



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Computer Vision

Computer Vision (S2-25_AIMLZG525)

Session #4: Edge Detection

Raghavendra Singh



BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Topics

- Edge Models & Approaches to edge detection
- Hough Transform - Detecting Lines

Acknowledgement: Slide Materials adopted from - Intro to Computer Vision (Cornell Tech); Noah Snavely

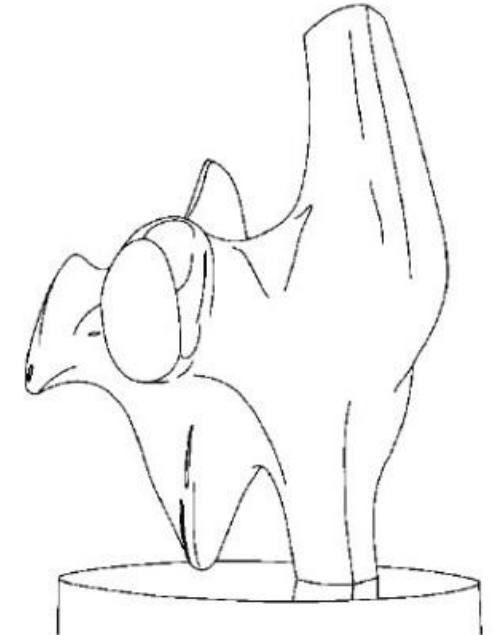
Edge Detection

Core Concept: What is Edge Detection?

The slide defines the primary task as: "**Convert a 2D image into a set of curves (made of edges).**"

In simple terms, it's about identifying the points in a digital image where the brightness changes sharply. These discontinuities usually correspond to:

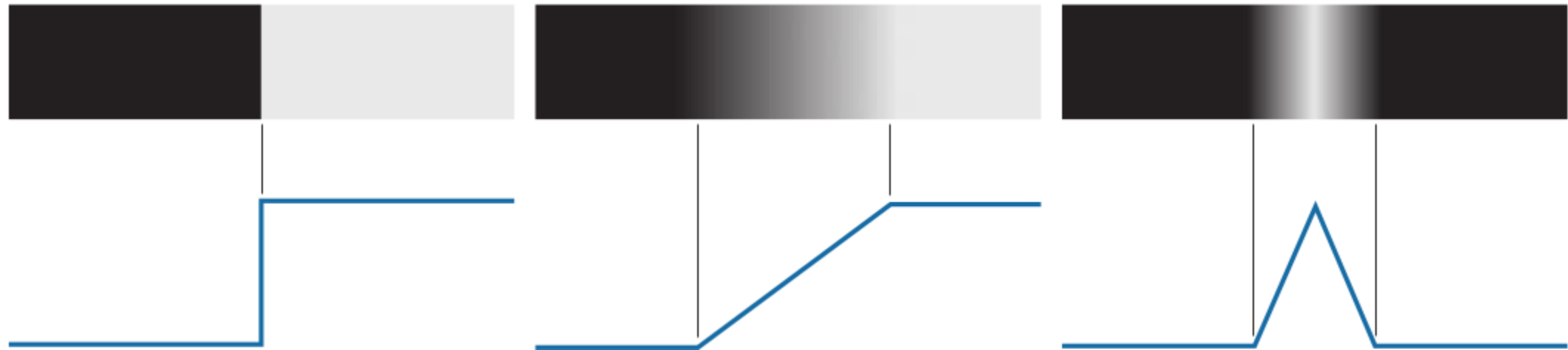
- **Discontinuities in depth:** Where one object ends and another (or the background) begins.
- **Discontinuities in surface orientation:** The "corners" or "ridges" of an object.
- **Changes in material properties:** Different colors or textures meeting.
- **Variations in scene illumination:** Such as shadows.



Task : Convert a 2D image into a set of curves (made of edges)



Edge Profiles



Step Edge

This slide illustrates **Edge Profiles**, which represent how the intensity (brightness) of pixels changes across different types of transitions in an image.

1. Step Edge

- **Visual:** An abrupt transition from a dark region (black) directly to a light region (white).
- **Profile:** The graph shows a sharp, instantaneous jump. This is the "ideal" edge often used in theoretical computer vision models.

Ramp Edge

2. Ramp Edge

- **Visual:** A gradual transition where the dark region slowly fades into the light region through a series of gray tones.
- **Profile:** The graph shows a diagonal slope (a ramp) rather than a vertical jump.

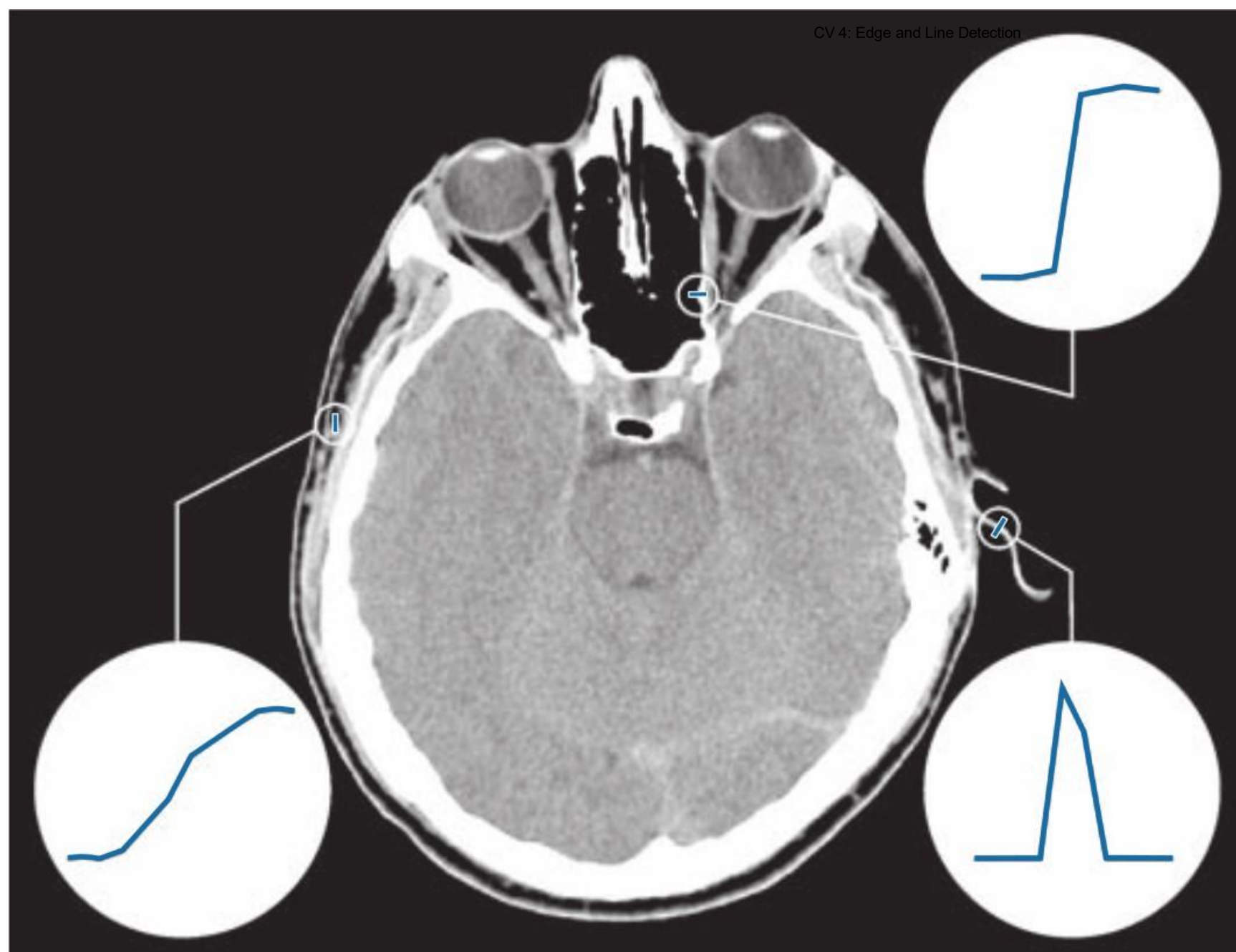
3. Roof Edge

- **Visual:** A thin, bright line (or a dark line) that stands out against its surroundings.
- **Profile:** The graph goes up and immediately back down, creating a triangle or "roof" shape.



Example

A 1508 1970 × image showing (zoomed) actual ramp (bottom, left), step (top, right), and roof edge profiles. The profiles are from dark to light, in the areas enclosed by the small circles. The ramp and step profiles span 9 pixels and 2 pixels, respectively. The base of the roof edge is 3 pixels. (Original image courtesy of Dr. David R. Pickens, Vanderbilt University.)





Recall: Edge Detector

We want an Edge Operator that produces:

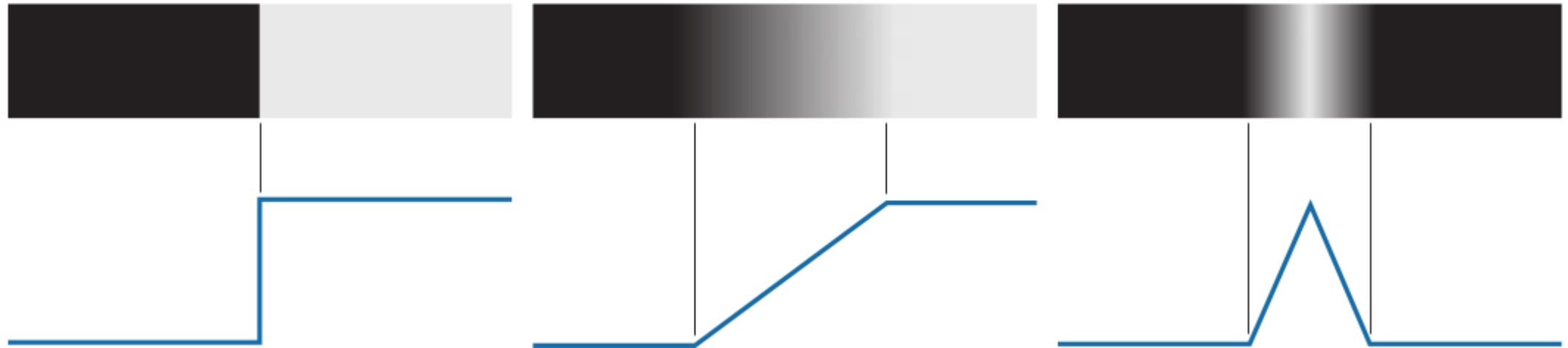
- Edge Position
- Edge Magnitude (Strength)
- Edge Orientation (Direction)

Performance Requirements:

- High Detection Rate
- Good Localization
- Low Noise Sensitivity



Recall: Edge Profiles

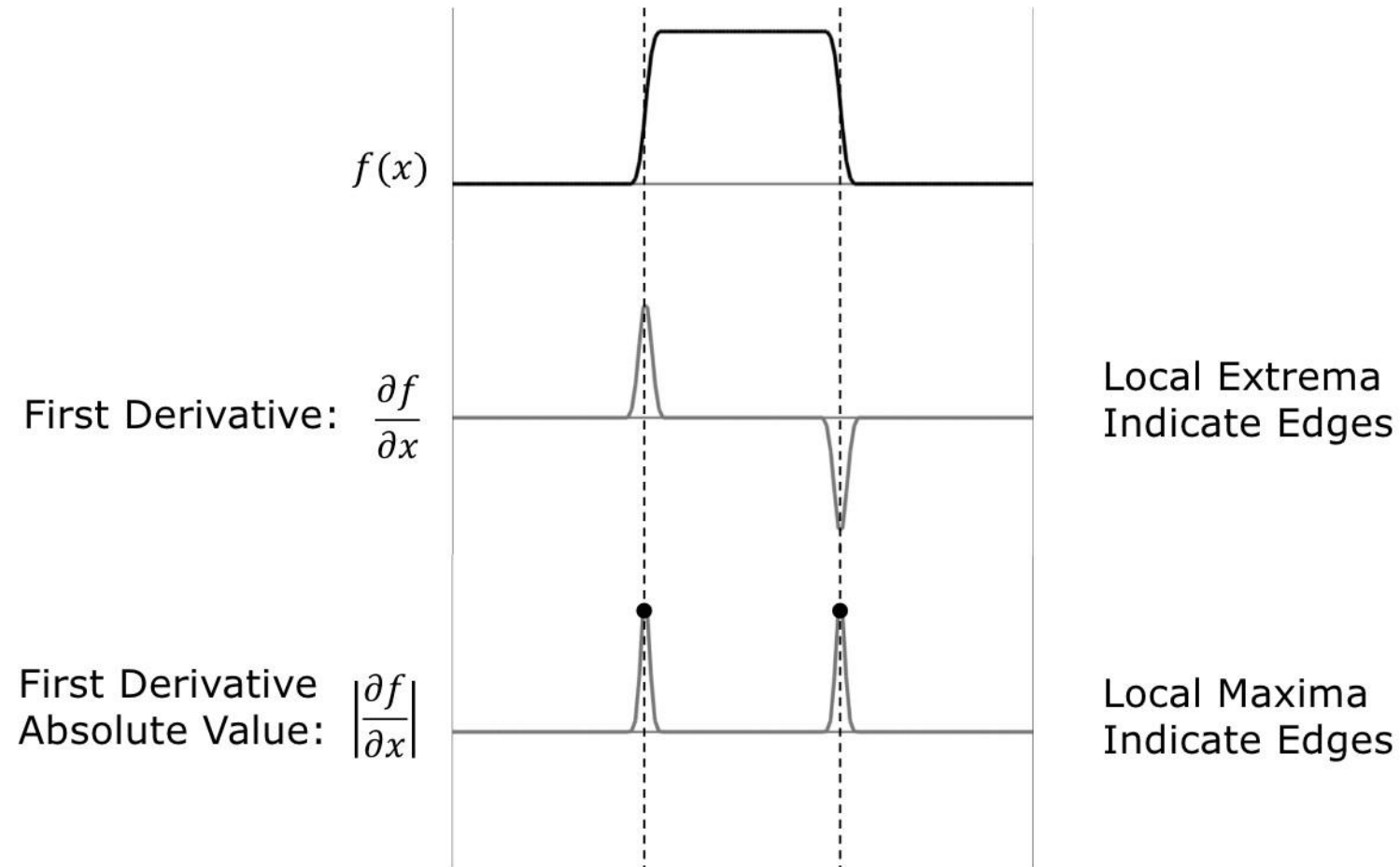


Step Edge

Roof Edge

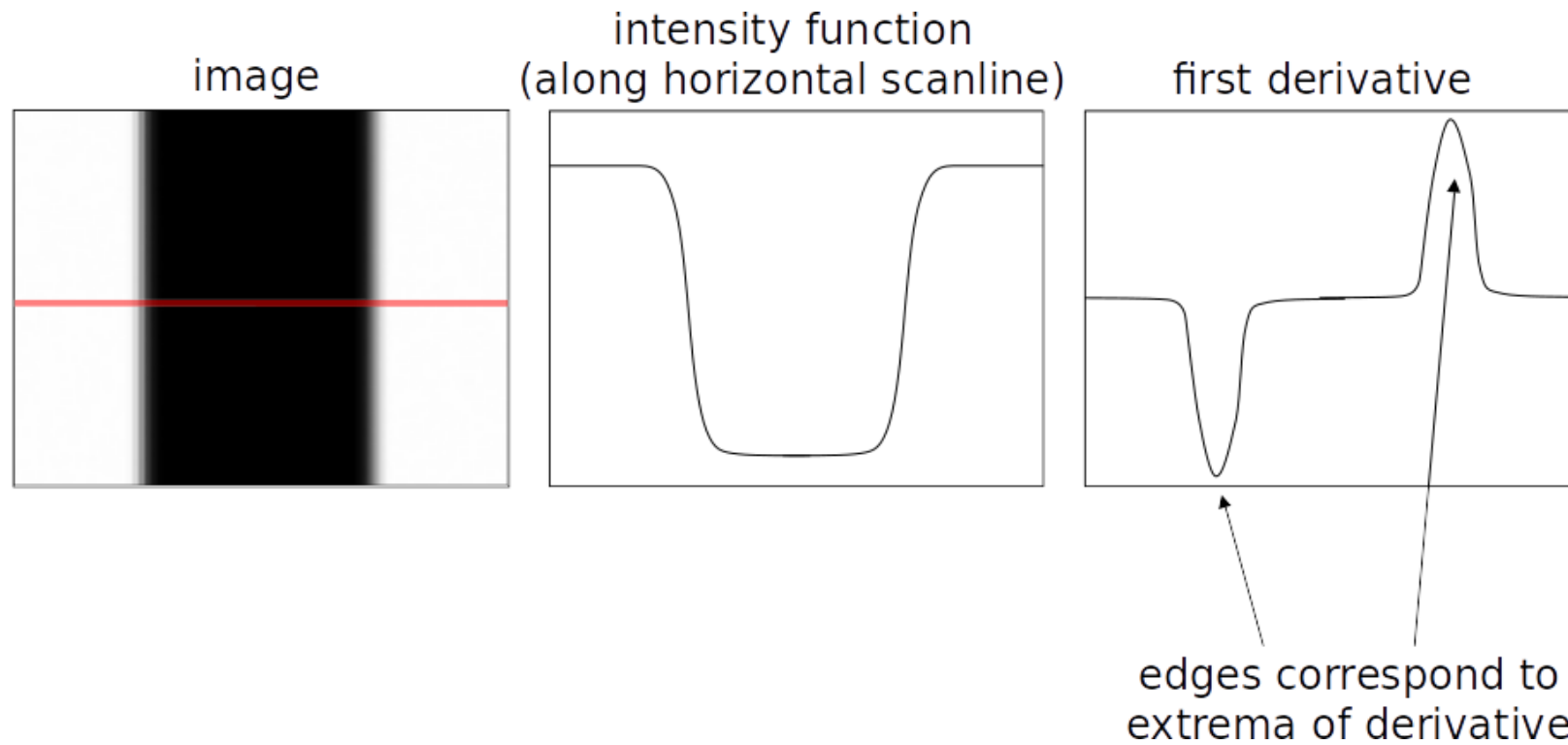
Ramp Edge

Recall: Edge detection using 1st Derivative



Provides Both Location and Strength of an Edge

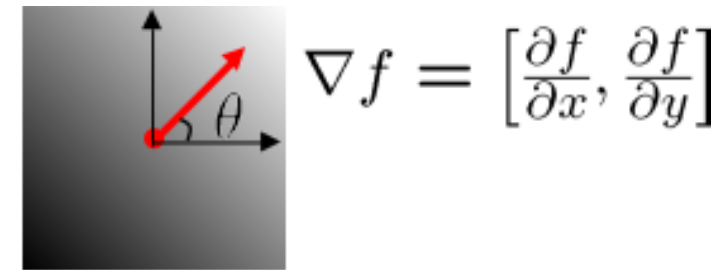
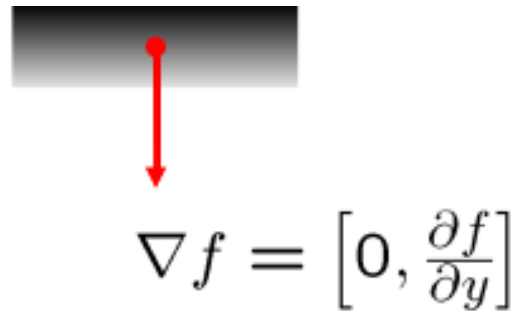
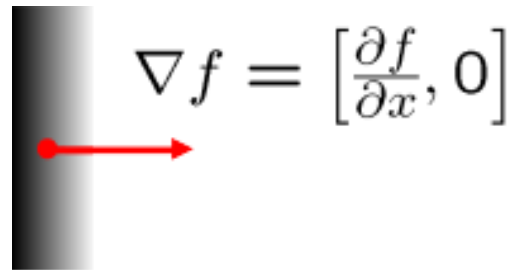
Recall: Edge Profiles



An edge is a place of rapid change in the image intensity function

Recall: Gradient's role in Detecting Edge

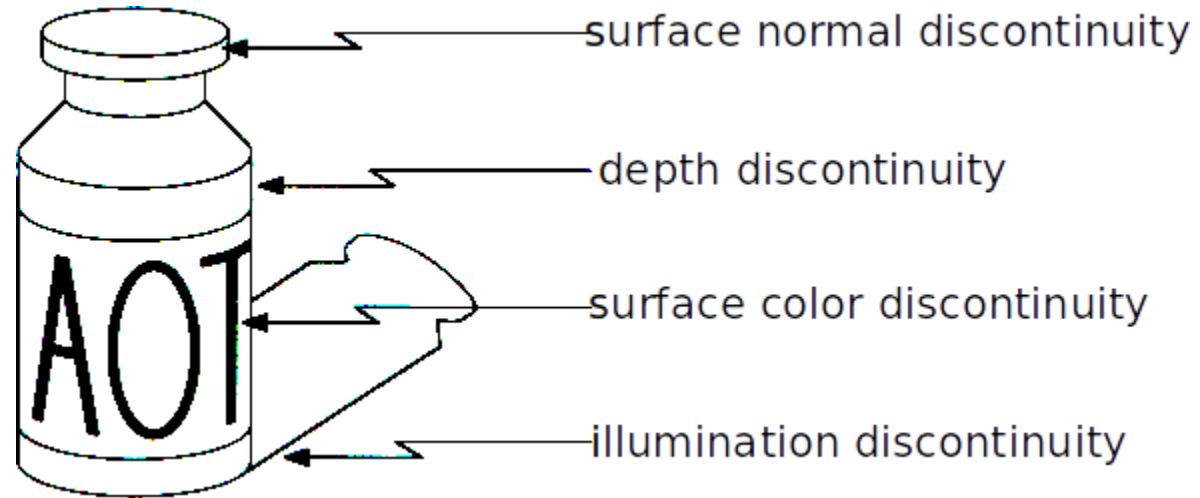
- The gradient of an image: $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$
- The gradient points in the *direction of most rapid increase in intensity*



- The edge strength is given by the gradient magnitude: $\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$
- The gradient direction is given by: $\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$

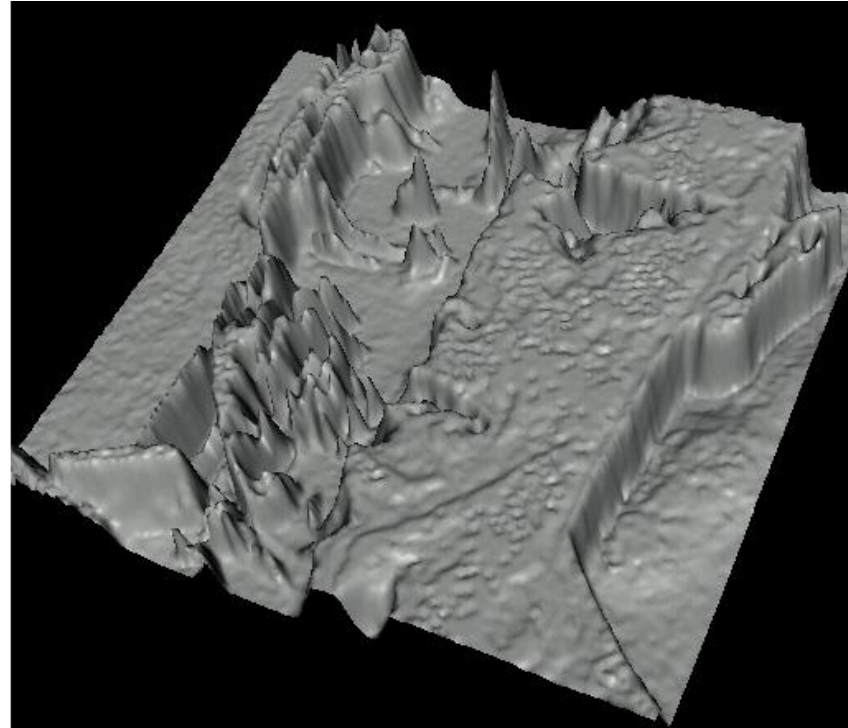


Edge Detection



Edges are caused by a variety of factors

Edge Detection



Treating images as functions, edges look like steep cliffs in the visualization



Recall: Gradient and it's Computation

- How can we differentiate a *digital* image $F[x,y]$?
 - **Option 1**: reconstruct a continuous image, f , then compute the derivative
 - **Option 2**: take discrete derivative (finite difference)

$$g_x(x, y) = \frac{\partial f(x, y)}{\partial x} = f(x + 1, y) - f(x, y)$$

$$g_y(x, y) = \frac{\partial f(x, y)}{\partial y} = f(x, y + 1) - f(x, y)$$

Comparing Gradient operators

Gradient	Roberts	Prewitt	Sobel (3x3)	Sobel (5x5)
$\frac{\partial I}{\partial x}$	0 1 -1 0	-1 0 1 -1 0 1 -1 0 1	-1 0 1 -2 0 2 -1 0 1	-1 -2 0 2 1 -2 -3 0 3 2 -3 -5 0 5 3 -2 -3 0 3 2 -1 -2 0 2 1
$\frac{\partial I}{\partial y}$	1 0 0 -1	1 1 1 0 0 0 -1 -1 -1	1 2 1 0 0 0 -1 -2 -1	1 2 3 2 1 2 3 5 3 2 0 0 0 0 0 -2 -3 -5 -3 -2 -1 -2 -3 -2 -1

Good Localization
Noise Sensitive
Poor Detection



Poor Localization
Less Noise Sensitive
Good Detection

Slide: Shree K Nayar



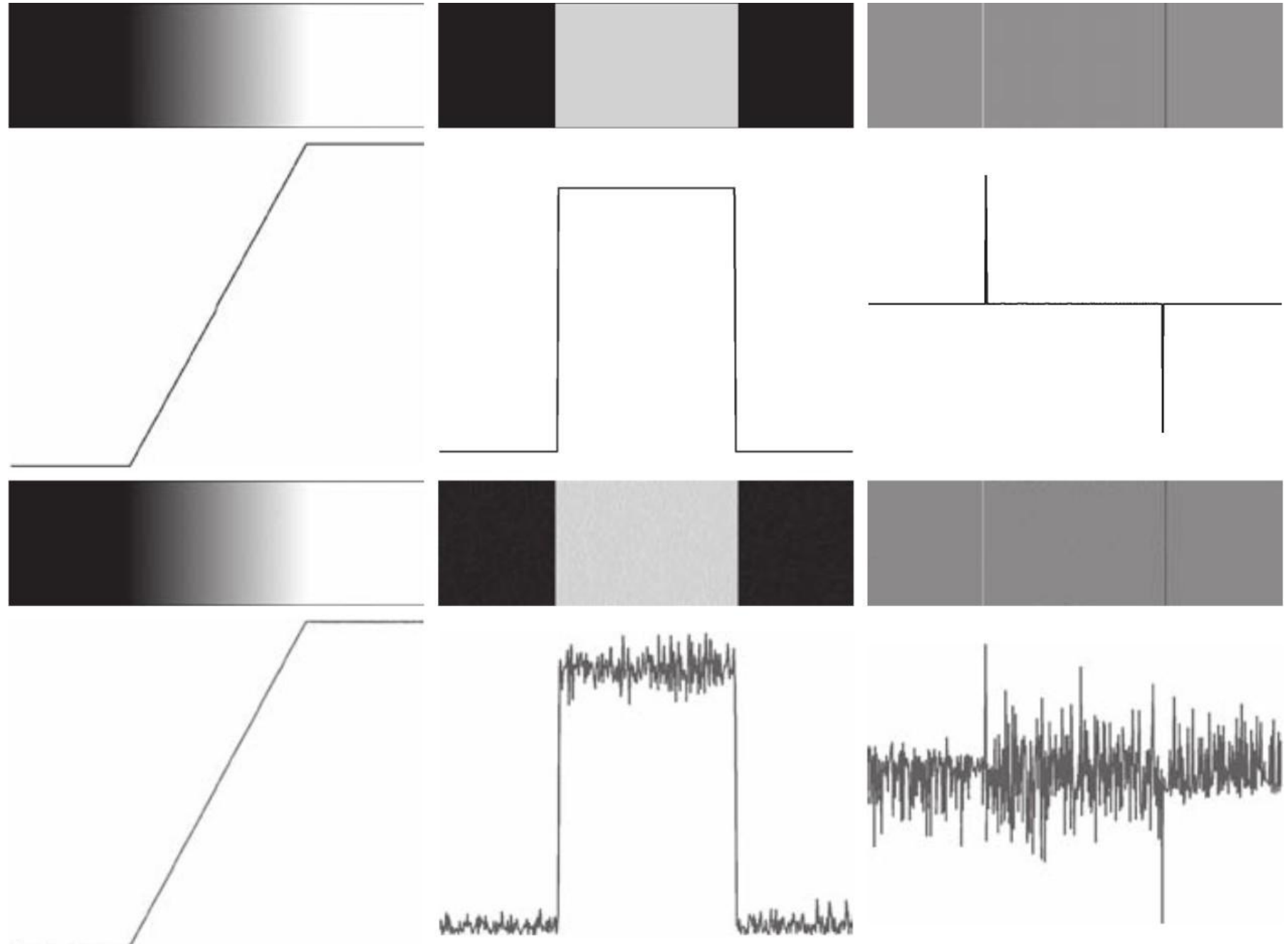
Steps in Canny's Edge Detection

1. Smooth the input image with a Gaussian filter.
2. Compute the gradient magnitude and angle images.
3. Apply non-maxima suppression to the gradient magnitude image.
4. Use double thresholding and connectivity analysis to detect and link edges.



Edge Detection Approach

First column: 8-bit images with values in the range $[0, 255]$ and intensity profiles of a ramp edge corrupted by Gaussian noise of zero mean and standard deviations of 0.0, 0.1, 1.0, and 10.0 intensity levels, respectively. Second column: First-derivative images and intensity profiles. Third column: Second-derivative images and intensity profiles.





Edge Detection Approach

1. **Image smoothing** for noise reduction.
2. **Detection of edge points**. As mentioned earlier, this is a local operation that extracts from an image all points that are potential edge-point candidates.
3. **Edge localization**. The objective of this step is to select from the candidate points only the points that are members of the set of points comprising an edge.

1. Image Smoothing

The Goal: Noise Reduction.

Digital images often contain "noise" (random speckles or graininess). If you try to find edges on a noisy image, the computer will mistake those tiny speckles for real edges.

- **How it works:** Usually involves applying a blur (like a Gaussian filter).

2. Detection of Edge Points

The Goal: Finding Candidates.

This is the "math-heavy" part. The computer looks for areas where the brightness changes suddenly (a high gradient).

- **How it works:** It uses operators (like Sobel, Prewitt, or Laplacian) to calculate the derivative of the pixel intensities.

3. Edge Localization

The Goal: Refinement and Selection.

Not every candidate point from Step 2 is a "true" edge. This step thins out the lines and discards the weak ones.

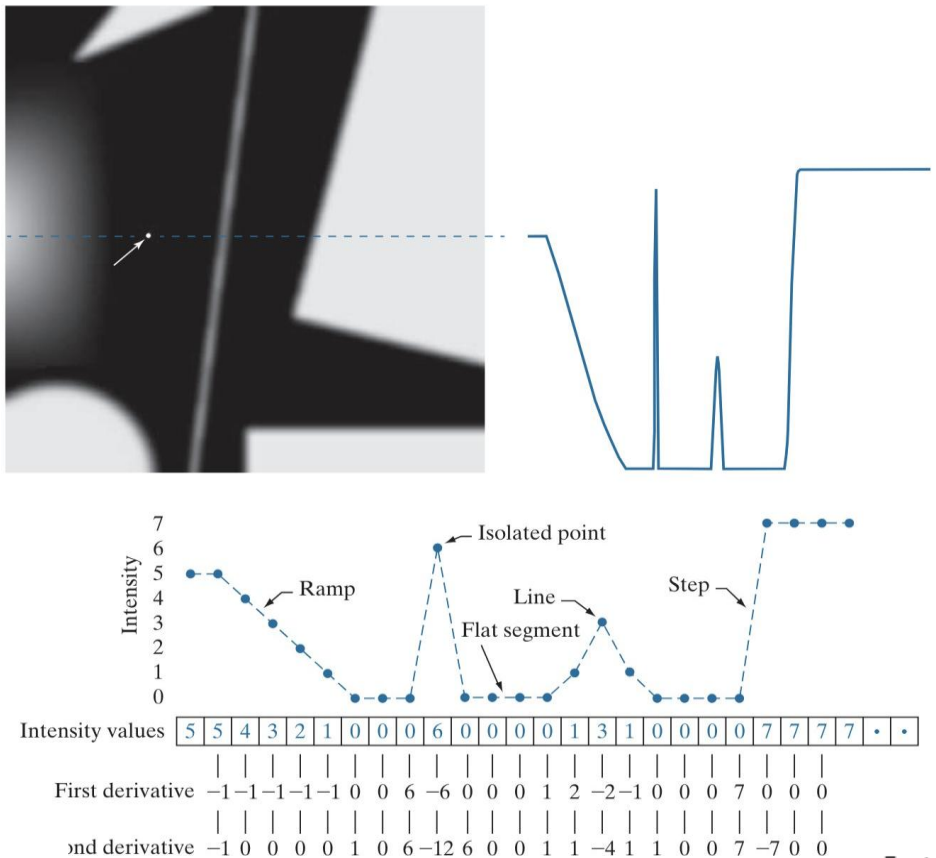
- **How it works:** * **Thinning:** Ensuring an edge is only one pixel wide (often called "Non-maximum suppression").
 - **Thresholding:** Setting a rule that says "only keep the strongest edges and throw away the rest."
- **Result:** A crisp, accurate set of lines representing the actual boundaries in the scene.

Why this order matters

If you skip **Step 1**, your result will be full of "false positives" (fake edges from noise). If you skip **Step 3**, your edges will look like thick, blurry smudges rather than distinct lines.

a b
c

FIGURE 10.2
(a) Image.
(b) Horizontal intensity profile that includes the isolated point indicated by the arrow.
(c) Subsampled profile; the dashes were added for clarity. The numbers in the boxes are the intensity values of the dots shown in the profile. The derivatives were obtained using Eqs. (10-4) for the first derivative and Eq. (10-7) for the second.



$$\frac{\partial f}{\partial x} = f'(x) = f(x+1) - f(x)$$

$$\frac{\partial^2 f}{\partial x^2} = f''(x) = f(x+1) + f(x-1) - 2f(x)$$

1. First Derivative (f')

The first derivative measures the **rate of change**. In digital images, this is calculated as the difference between adjacent pixels:

$$\frac{\partial f}{\partial x} = f(x+1) - f(x)$$

2. Second Derivative (f'')

The second derivative measures the **change in the rate of change** (curvature):

$$\frac{\partial^2 f}{\partial x^2} = f(x+1) + f(x-1) - 2f(x)$$

- **Behavior:** It produces a non-zero value along a ramp or at a step.
- **Use Case:** It is primarily used to detect the **presence** of an edge.

- **Behavior:** It produces a "double response" (one positive, one negative) at the beginning and end of ramps or lines.
- **Zero-Crossing:** Crucially, it crosses through zero in the middle of a transition, which is used to pinpoint the exact center of an edge.

Feature	First Derivative Response	Second Derivative Response
Flat Segment	Zero	Zero
Isolated Point	Large positive/negative values	Very large response (stronger than 1st)
Line	Responds at both boundaries	Double response at both boundaries
Step/Edge	Non-zero during the transition	Positive at start, negative at end



Gradient and it's Computation

- How can we differentiate a *digital* image $F[x,y]$?
 - **Option 1:** reconstruct a continuous image, f , then compute the derivative
 - **Option 2:** take discrete derivative (finite difference)

1. Horizontal Gradient (g_x)

$$g_x(x, y) = \frac{\partial f(x, y)}{\partial x} = f(x + 1, y) - f(x, y)$$

$$g_x(x, y) = \frac{\partial f(x, y)}{\partial x} = f(x + 1, y) - f(x, y)$$
$$g_y(x, y) = \frac{\partial f(x, y)}{\partial y} = f(x, y + 1) - f(x, y)$$

- This measures how the intensity changes as you move one pixel to the **right**.

2. Vertical Gradient (g_y)

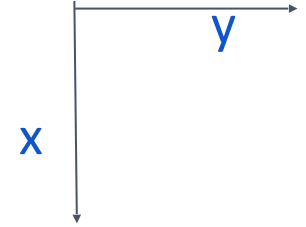
$$g_y(x, y) = \frac{\partial f(x, y)}{\partial y} = f(x, y + 1) - f(x, y)$$

- This measures how the intensity changes as you move one pixel **down**.

Why does this matter?

In image processing, the "Gradient" combines these two values to tell us:

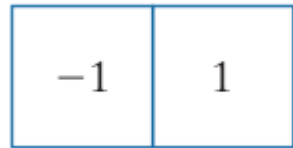
1. **Magnitude:** How "strong" an edge is (the contrast).
2. **Direction:** Which way the edge is pointing.



Gradient and it's Computation

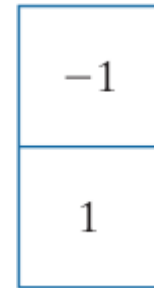
- How can we differentiate a *digital* image $F[x,y]$?
 - Option 1:** reconstruct a continuous image, f , then compute the derivative
 - Option 2:** take discrete derivative (finite difference)

$$g_x(x, y) = \frac{\partial f(x, y)}{\partial x} = f(x + 1, y) - f(x, y)$$



(a)

$$g_y(x, y) = \frac{\partial f(x, y)}{\partial y} = f(x, y + 1) - f(x, y)$$



(b)

Derivative Kernels (Masks)

The boxes labeled **(a)** and **(b)** are the "kernels" or "filters" used to perform these calculations efficiently in software:

- Filter (a) for g_x :** Represented by the values $[-1, 1]$. When you place this over two horizontal pixels, it subtracts the first from the second, exactly matching the formula above.
- Filter (b) for g_y :** Represented by the values $\begin{bmatrix} -1 \\ 1 \end{bmatrix}$. This performs the same subtraction but in the vertical direction.



Gradient and it's Computation

Compute partial derivatives $\partial f/\partial x$ and $\partial f/\partial y$ at every pixel location in the image

$$\nabla f(x, y) \equiv \text{grad}[f(x, y)] \equiv \begin{bmatrix} g_x(x, y) \\ g_y(x, y) \end{bmatrix} = \begin{bmatrix} \frac{\partial f(x, y)}{\partial x} \\ \frac{\partial f(x, y)}{\partial y} \end{bmatrix}$$

Points in the direction of maximum rate of change of f at (x, y)

$$\alpha(x, y) = \tan^{-1} \left[\frac{g_y(x, y)}{g_x(x, y)} \right]$$

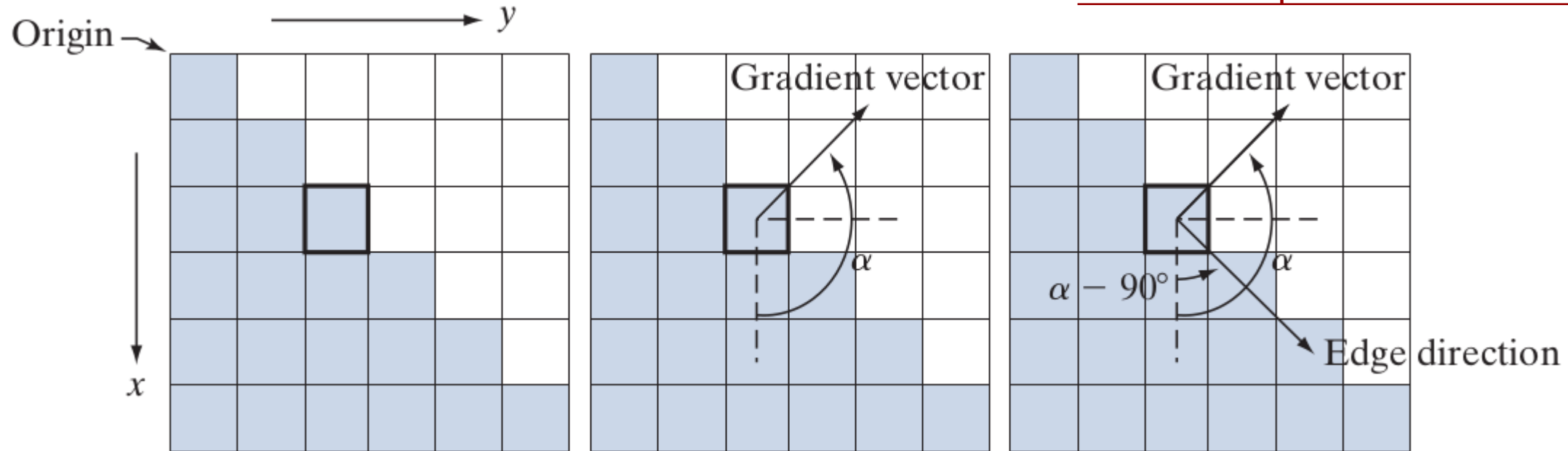
Direction of gradient vector at (x, y)

$$M(x, y) = \|\nabla f(x, y)\| = \sqrt{g_x^2(x, y) + g_y^2(x, y)}$$

Value of the rate of change in the direction of the gradient vector at (x, y)

Gradient and it's Computation - Example

The shaded pixels are 0's & white are 2's

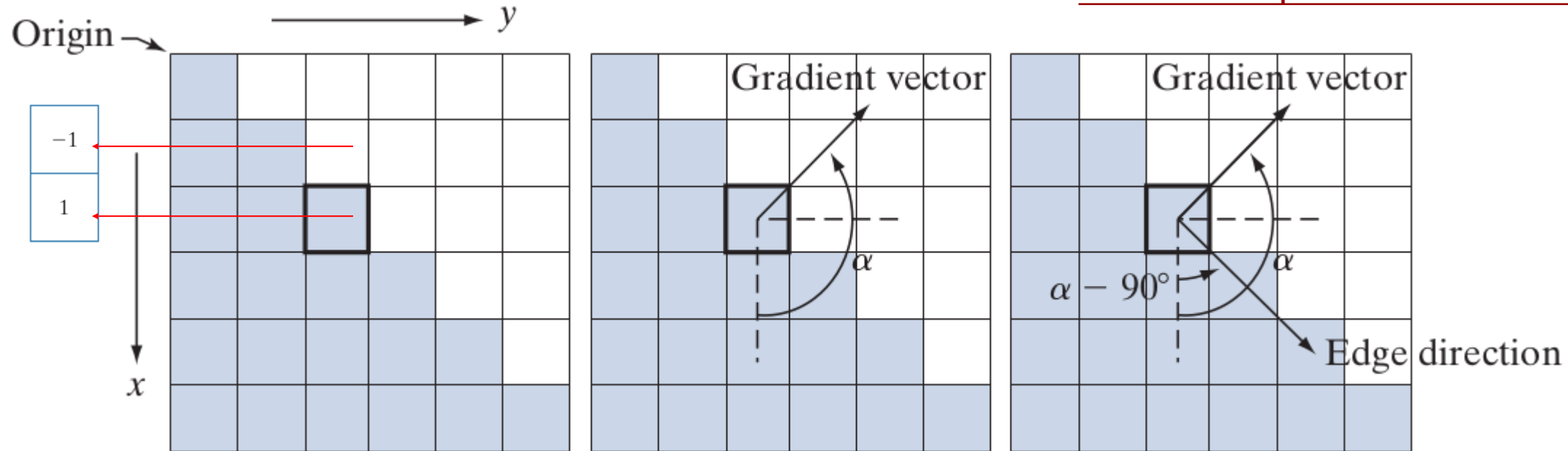


$$g_x(x, y) = \frac{\partial f(x, y)}{\partial x} = f(x + 1, y) - f(x, y)$$

$$g_y(x, y) = \frac{\partial f(x, y)}{\partial y} = f(x, y + 1) - f(x, y)$$

Gradient and it's Computation - Example

The shaded pixels are 0's & white are 2's



$$g_x(x, y) = \frac{\partial f(x, y)}{\partial x} = f(x + 1, y) - f(x, y)$$

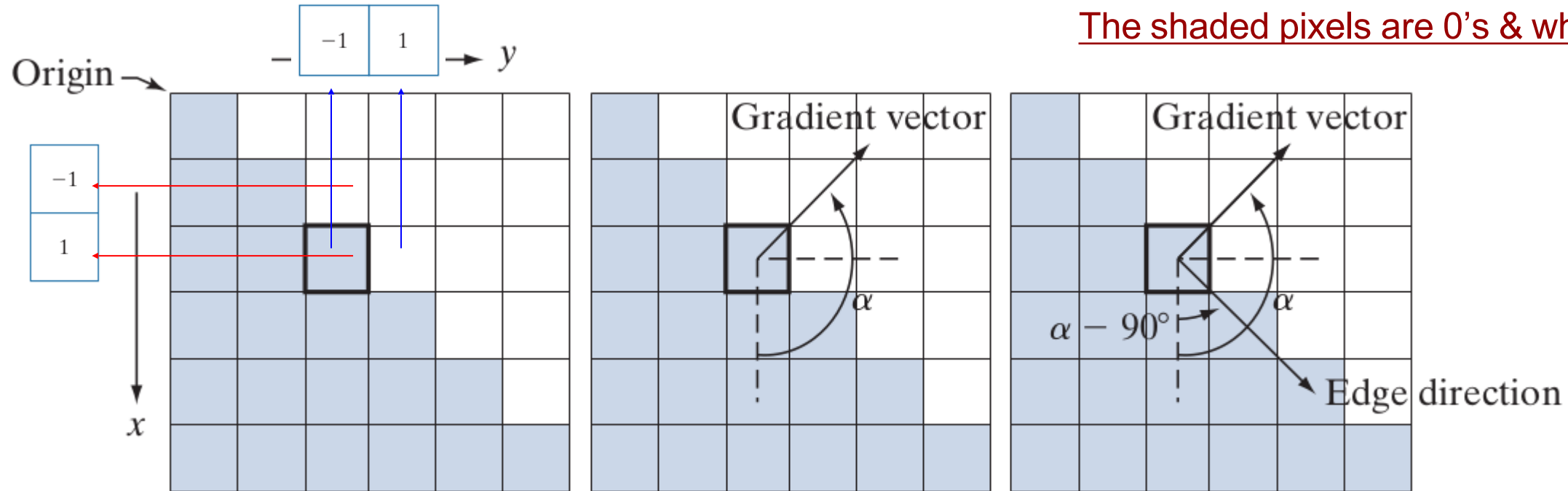
Subtract pixels in the top row of the neighborhood from the pixels in the bottom row → partial derivative in the x-direction

$$g_y(x, y) = \frac{\partial f(x, y)}{\partial y} = f(x, y + 1) - f(x, y)$$

Subtract pixels in the left column from the pixels in the right column of the neighborhood → partial derivative in the y-direction

Gradient and it's Computation - Example

The shaded pixels are 0's & white are 2's



$$g_x(x, y) = \frac{\partial f(x, y)}{\partial x} = f(x + 1, y) - f(x, y) = -2$$

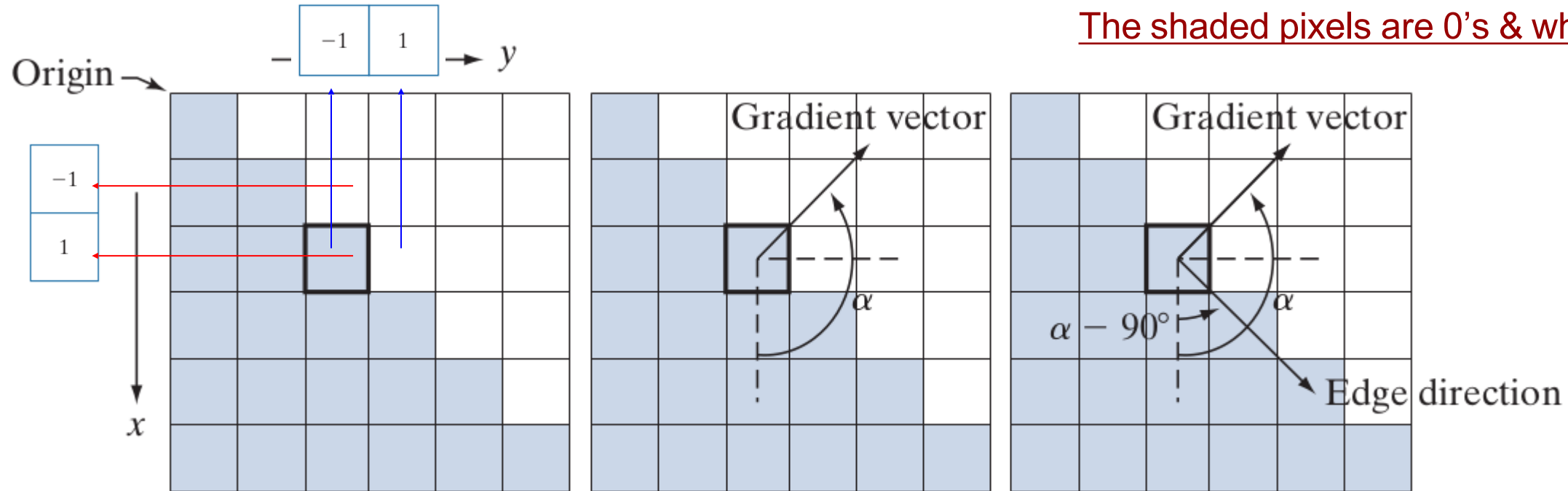
$$g_y(x, y) = \frac{\partial f(x, y)}{\partial y} = f(x, y + 1) - f(x, y) = 2$$



$$\nabla f = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} -2 \\ 2 \end{bmatrix}$$

Gradient and it's Computation - Example

The shaded pixels are 0's & white are 2's

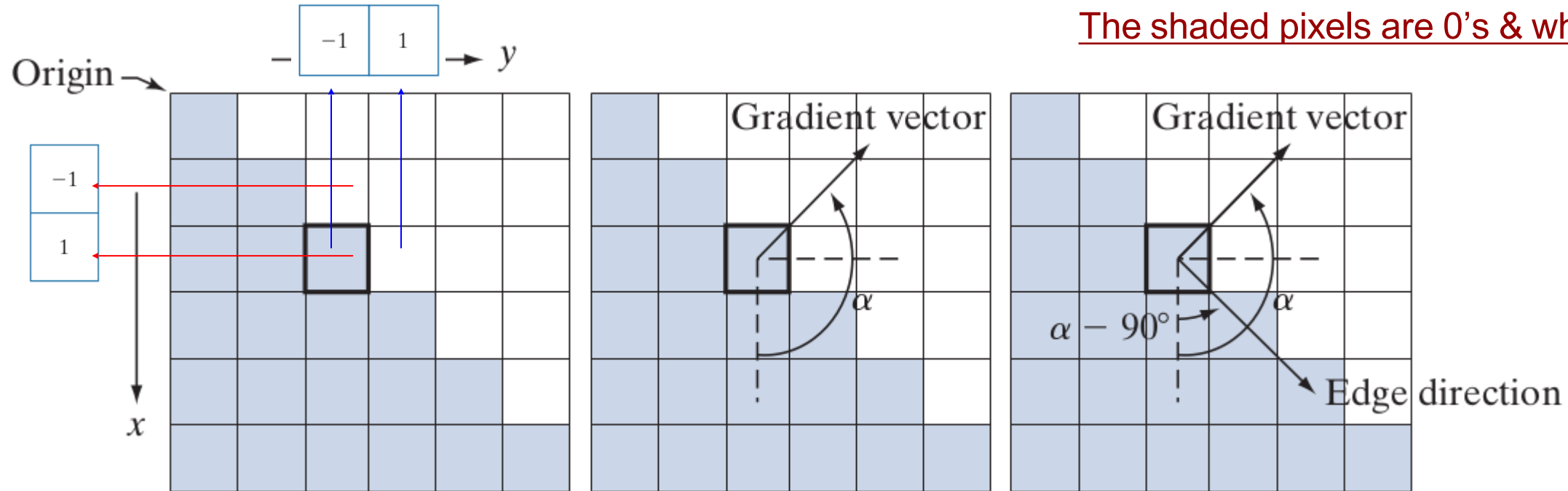


$$\|\nabla f\| = 2\sqrt{2}$$

$$\alpha = \tan^{-1}(g_y/g_x) = -45^\circ = 135^\circ$$

$$\nabla f = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} -2 \\ 2 \end{bmatrix}$$

Gradient and it's Computation - Example



Direction of an edge at a point is orthogonal to the gradient vector at that point → Direction of Edge is

$$\alpha - 90^\circ = 135^\circ - 90^\circ = 45^\circ$$

$$\alpha = \tan^{-1}(g_y/g_x) = -45^\circ = 135^\circ$$

$$\nabla f = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} -2 \\ 2 \end{bmatrix}$$



2. Calculating the Change (The Formulas)

To find where an edge is, we look at the difference between neighboring pixels. This is called the **partial derivative** (∂).

- $g_x(x, y)$ (**Vertical Change**): It subtracts the current pixel from the one below it. If there is a change in color as you move down, g_x will be non-zero.

$$g_x(x, y) = \frac{\partial f(x, y)}{\partial x} = f(x + 1, y) - f(x, y)$$

- $g_y(x, y)$ (**Horizontal Change**): It subtracts the current pixel from the one to its right. If there is a change as you move right, g_y will be non-zero.

$$g_y(x, y) = \frac{\partial f(x, y)}{\partial y} = f(x, y + 1) - f(x, y)$$

- **Gradient Vector**: This vector points in the direction of the **steepest increase** in intensity (from dark to light). In the diagram, it points toward the white area.
- **Angle (α)**: This represents the direction of the gradient vector.
- **Edge Direction**: Crucially, the actual edge in the image is always **perpendicular** (90° offset) to the gradient vector. While the gradient points "up the hill," the edge runs "along the ridge."

1. The Setup: Pixels as Numbers

The grid represents a digital image where:

- **Shaded pixels (blue/grey)** have a value of **0** (darker).
- **White pixels** have a value of **2** (lighter).
- The **Origin** is at the top-left, with x increasing downwards and y increasing to the right.

1. Identify the Values

Let's look at the bolded pixel and its immediate neighbors to the right and below:

- **Current Pixel** $f(x, y)$: This is shaded, so its value is **0**.
- **Pixel to the Right** $f(x, y + 1)$: This is white, so its value is **2**.
- **Pixel Below** $f(x + 1, y)$: This is white, so its value is **2**.

2. Calculate the Partial Derivatives

Now we plug these into the formulas provided at the bottom of the slide:

Horizontal Change (g_y):

$$g_y(x, y) = f(x, y + 1) - f(x, y) = 2 - 0 = 2$$

Interpretation: The brightness increases by 2 units as we move to the right.

Vertical Change (g_x):

$$g_x(x, y) = f(x + 1, y) - f(x, y) = 2 - 0 = 2$$

Interpretation: The brightness increases by 2 units as we move down.



3. Determine the Gradient Vector

The Gradient Vector ∇f is composed of these two values: $\begin{bmatrix} 2 \\ 2 \end{bmatrix}$.

- **Direction (α):** Since both g_x and g_y are equal (2 and 2), the vector points exactly at a **45° angle** between the x-axis (down) and y-axis (right). This matches the diagonal arrow shown in the "Gradient vector" diagram.
- **Magnitude:** This tells us how "strong" the edge is. We calculate it using the Pythagorean theorem:

$$\text{Magnitude} = \sqrt{g_x^2 + g_y^2} = \sqrt{2^2 + 2^2} = \sqrt{8} \approx 2.83$$

4. Finding the Edge

As the third diagram shows, the **Edge direction** is perpendicular to this gradient.

- If the gradient points at **45°** (down-right), the edge itself runs at **-45°** (up-right to down-left).
- This perfectly traces the diagonal "staircase" boundary seen in the image.



Earlier Numerical using Sobel operator ! Extra !!

Why use Sobel instead?

The simple method in your slide is very sensitive to **noise**. If a single pixel is "off" due to a camera glitch, the gradient will be wrong. The Sobel operator uses a weighted average of surrounding pixels, which "smooths" the image slightly and makes the edge detection much more reliable.

1. The Sobel Kernels (The Math Masks)

Instead of a simple formula, we use two matrices (kernels) that "slide" over the image.

- G_x (**Vertical/X-gradient**): Detects horizontal edges by comparing pixels above the center to pixels below the center.
- G_y (**Horizontal/Y-gradient**): Detects vertical edges by comparing pixels to the left of the center to pixels to the right.

The number **2** in the middle of the mask gives more "importance" to the pixel directly adjacent to the center.

2. Numerical Example (The 3x3 Window)

Let's look at a 3x3 block of pixels centered on the bolded pixel from your slide. Based on the "staircase" pattern:

(x-1, y-1)	(x-1, y)	(x-1, y+1)
0 (Shaded)	0 (Shaded)	2 (White)
0 (Shaded)	0 (Center)	2 (White)
0 (Shaded)	2 (White)	2 (White)

Calculating G_y (Horizontal Change)

We multiply the 3x3 grid by the G_y Sobel kernel and sum them up:

$$G_y = (0 \cdot -1) + (0 \cdot 0) + (2 \cdot 1) + (0 \cdot -2) + (0 \cdot 0) + (2 \cdot 2) + (0 \cdot -1) + (2 \cdot 0) + (2 \cdot 0)$$

$$G_y = 0 + 0 + 2 + 0 + 0 + 4 + 0 + 0 + 2 = 8$$

Calculating G_x (Vertical Change)

We do the same with the G_x kernel:

$$G_x = (0 \cdot -1) + (0 \cdot -2) + (2 \cdot -1) + (0 \cdot 0) + (0 \cdot 0) + (2 \cdot 0) + (0 \cdot 1) + (2 \cdot 2) + (2 \cdot 0)$$

$$G_x = 0 + 0 - 2 + 0 + 0 + 0 + 0 + 4 + 2 = 4$$



Using Sobel operator

3. Comparing the Results

Method	G_x	G_y	Direction ($\tan^{-1}(G_x/G_y)$)
Slide Method	2	2	45°
Sobel Method	4	8	~26.5°

Why the difference?

The Sobel operator "sees" more of the neighborhood. In our 3x3 window, there are more white pixels to the right than there are white pixels below. Therefore, the Sobel operator decides the gradient is pulling more strongly to the **right** ($G_y = 8$) than it is **down** ($G_x = 4$).

This gives a much more accurate representation of the "true" slope of the edge across that specific area of the image.



Gradient and it's Computation - Different Forms

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

A 3×3 region of an image (the z 's are intensity values)

To Do: Compute gradient at z_5



Gradient and it's Computation - Different Forms

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

Prewitt's operators

-1	-1	-1
0	0	0
1	1	1

-1	0	1
-1	0	1
-1	0	1

A 3×3 region of an image (the z's are intensity values)

To Do: Compute gradient at z_5

$$g_x = \frac{\partial f}{\partial x} = (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)$$

$$g_y = \frac{\partial f}{\partial y} = (z_3 + z_6 + z_9) - (z_1 + z_4 + z_7)$$



1. Compute the Horizontal Component (g_x)

This operator detects horizontal edges by calculating the difference between the **bottom row** and the **top row**.

- **Positive Weight:** The bottom row (z_7, z_8, z_9) is multiplied by $+1$.
- **Negative Weight:** The top row (z_1, z_2, z_3) is multiplied by -1 .
- **Zero Weight:** The middle row (z_4, z_5, z_6) is ignored.

$$g_x = (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)$$

2. Compute the Vertical Component (g_y)

This operator detects vertical edges by calculating the difference between the **right column** and the **left column**.

- **Positive Weight:** The right column (z_3, z_6, z_9) is multiplied by $+1$.
- **Negative Weight:** The left column (z_1, z_4, z_7) is multiplied by -1 .
- **Zero Weight:** The center column (z_2, z_5, z_8) is ignored.

$$g_y = (z_3 + z_6 + z_9) - (z_1 + z_4 + z_7)$$

3. Calculate the Gradient Magnitude

Once you have the numerical values for g_x and g_y , you combine them to find the total "strength" of the gradient at z_5 . This tells you how sharp the edge is at that specific pixel.

The standard formula is the Euclidean norm:

$$M(z_5) = \sqrt{g_x^2 + g_y^2}$$

Pro Tip: In many real-time applications, engineers use the approximation $M(z_5) \approx |g_x| + |g_y|$ because it is much faster for a computer to calculate than a square root.

4. Determine the Gradient Direction

If you need to know which way the edge is pointing (the angle α), you use the arctangent of the two components:

$$\alpha(z_5) = \tan^{-1} \left(\frac{g_y}{g_x} \right)$$



Gradient and it's Computation - Different Forms

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

Sobel's operators

-1	-2	-1
0	0	0
1	2	1

-1	0	1
-2	0	2
-1	0	1

A 3×3 region of an image (the z 's are intensity values)

$$g_x = \frac{\partial f}{\partial x} = (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)$$

To Do: Compute gradient at z_5

$$g_y = \frac{\partial f}{\partial y} = (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)$$



Gradient and it's Computation - How to use?

- (1) Use any of the gradient operators to obtain g_x and g_y at every pixel location
- (2) Approximate gradient at each of the pixel locations as below:

$$M(x, y) \approx |g_x| + |g_y|$$

Gradient and it's Computation - Example

a	b
c	d

(a) Image of size 834×1114 pixels, with intensity values scaled to $[0, 1]$.

(b) $|g_x|$, the component of gradient obtained using the Sobel kernel

(c) $|g_y|$

(d) The gradient (using sum of absolute values)





Gradient and it's Computation - Example

a	b
c	d

Same sequence as prev. fig. but with the original image smoothed using a 5×5 averaging kernel prior to edge detection.





Gradient and it's Computation - Kirsch compass kernels

<table><tr><td>-3</td><td>-3</td><td>5</td></tr><tr><td>-3</td><td>0</td><td>5</td></tr><tr><td>-3</td><td>-3</td><td>5</td></tr></table> N	-3	-3	5	-3	0	5	-3	-3	5	<table><tr><td>-3</td><td>5</td><td>5</td></tr><tr><td>-3</td><td>0</td><td>5</td></tr><tr><td>-3</td><td>-3</td><td>-3</td></tr></table> NW	-3	5	5	-3	0	5	-3	-3	-3	<table><tr><td>5</td><td>5</td><td>5</td></tr><tr><td>-3</td><td>0</td><td>-3</td></tr><tr><td>-3</td><td>-3</td><td>-3</td></tr></table> W	5	5	5	-3	0	-3	-3	-3	-3	<table><tr><td>5</td><td>5</td><td>-3</td></tr><tr><td>5</td><td>0</td><td>-3</td></tr><tr><td>-3</td><td>-3</td><td>-3</td></tr></table> SW	5	5	-3	5	0	-3	-3	-3	-3
-3	-3	5																																					
-3	0	5																																					
-3	-3	5																																					
-3	5	5																																					
-3	0	5																																					
-3	-3	-3																																					
5	5	5																																					
-3	0	-3																																					
-3	-3	-3																																					
5	5	-3																																					
5	0	-3																																					
-3	-3	-3																																					
<table><tr><td>5</td><td>-3</td><td>-3</td></tr><tr><td>5</td><td>0</td><td>-3</td></tr><tr><td>5</td><td>-3</td><td>-3</td></tr></table> S	5	-3	-3	5	0	-3	5	-3	-3	<table><tr><td>-3</td><td>-3</td><td>-3</td></tr><tr><td>5</td><td>0</td><td>-3</td></tr><tr><td>5</td><td>5</td><td>-3</td></tr></table> SE	-3	-3	-3	5	0	-3	5	5	-3	<table><tr><td>-3</td><td>-3</td><td>-3</td></tr><tr><td>-3</td><td>0</td><td>-3</td></tr><tr><td>5</td><td>5</td><td>5</td></tr></table> E	-3	-3	-3	-3	0	-3	5	5	5	<table><tr><td>-3</td><td>-3</td><td>-3</td></tr><tr><td>-3</td><td>0</td><td>5</td></tr><tr><td>-3</td><td>5</td><td>5</td></tr></table> NE	-3	-3	-3	-3	0	5	-3	5	5
5	-3	-3																																					
5	0	-3																																					
5	-3	-3																																					
-3	-3	-3																																					
5	0	-3																																					
5	5	-3																																					
-3	-3	-3																																					
-3	0	-3																																					
5	5	5																																					
-3	-3	-3																																					
-3	0	5																																					
-3	5	5																																					



This slide illustrates the **Kirsch Compass Kernels**, which are a specific type of non-linear edge detector used in image processing. Unlike a simple Sobel filter that usually only looks for horizontal and vertical edges, the Kirsch operator is designed to find the **maximum edge strength** and its **orientation**.

Here is a breakdown of how these work:

1. The "Compass" Concept

The operator uses eight different 3×3 kernels (filters). Each one is sensitive to an edge in specific direction, named after the points on a compass:

- **N, NW, W, SW, S, SE, E, NE**

By rotating the positive weights (**5**) and negative weights (**-3**) around the center, the filter "looks" for edges at 45° increments.

2. How the Math Works

To process an image using these kernels, the computer follows these steps:

1. **Convolution:** The image is convolved with all **eight** kernels separately.
2. **Magnitude:** For every single pixel, the response from each of the eight kernels is calculated. The final edge magnitude at that pixel is the **maximum** value produced by any of the kernels:

$$M(x, y) = \max_{i=1 \dots 8} \{|h_i * I(x, y)|\}$$

3. **Direction:** The direction of the edge is defined by which specific kernel produced that maximum value.

3. Key Characteristics

- **Zero Sum:** If you add up all the numbers in any of these kernels, the sum is **0** ($5 \times 3 = 15$ and $-3 \times 5 = -15$). This means the filter ignores areas of constant brightness (flat regions) and only reacts when there is a change in intensity.
- **Symmetry:** You'll notice the kernels are just rotated versions of each other. For example, the **North (N)** kernel has the "5"s on the right, meaning it is most sensitive to vertical edges where the intensity changes from left to right.
- **Robustness:** Because it checks 8 directions, it is much more accurate at finding diagonal edges than basic gradient operators.

Why use this over other filters?

While standard filters like Sobel or Prewitt are faster (requiring only 2 kernels), the Kirsch operator is more **directional**. It's particularly useful when you need to know the exact orientation of a boundary, such as in fingerprint analysis or medical imaging.

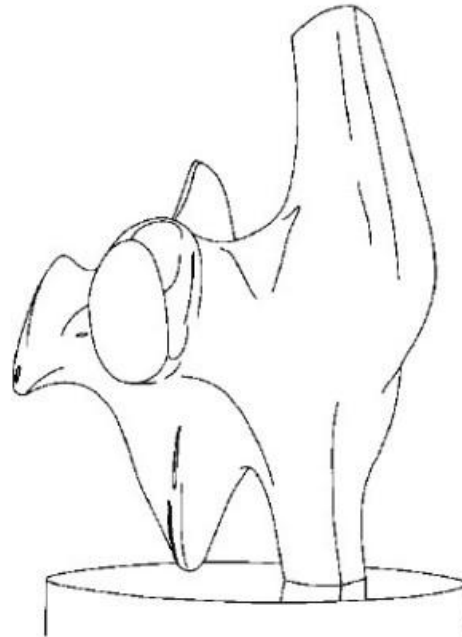


Canny Edge Detection

A powerful Edge Detection Approach,
Deal with it in the Next Class

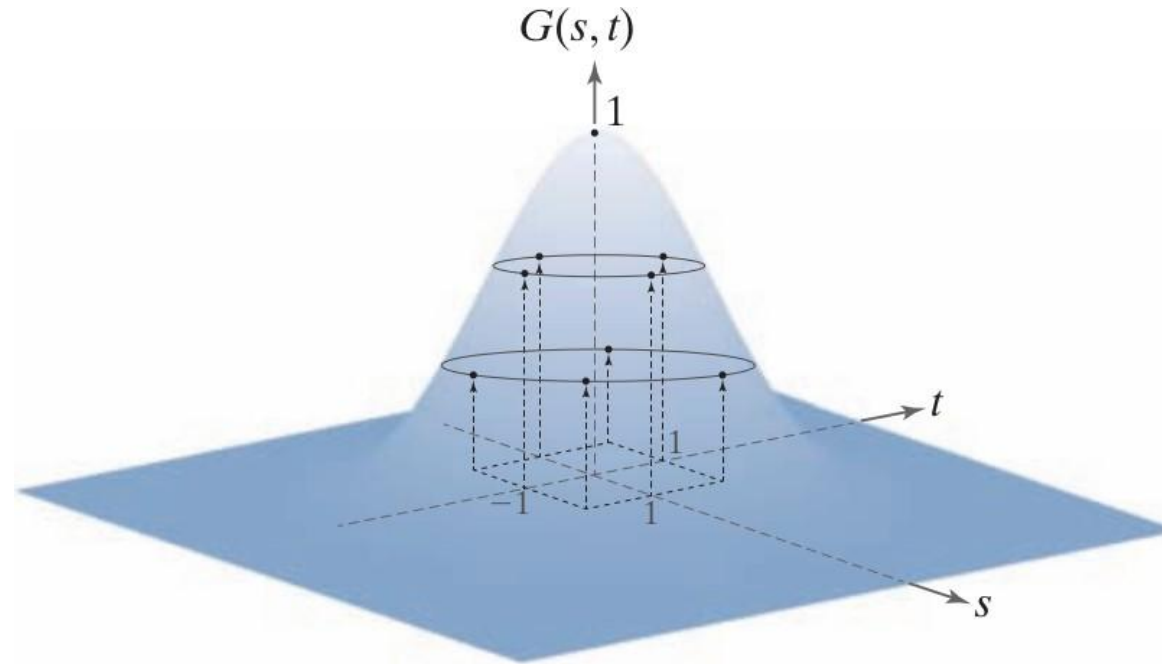
Image Sharpening - Next Class

Edge Detection



Task : Convert a 2D image into a set of curves (made of edges)

Recall: Gaussian Filters



$$\frac{1}{4.8976} \times$$

0.3679	0.6065	0.3679
0.6065	1.0000	0.6065
0.3679	0.6065	0.3679

Resulting 3 × 3 kernel

$$r = [s^2 + t^2]^{1/2}$$

$$G(r) = Ke^{-\frac{r^2}{2\sigma^2}}$$

Sampling a Gaussian function to obtain a discrete Gaussian kernel. The values shown are for K = 1 and $\sigma = 1$.

Recall: Gaussian Filters



A test pattern of size 1024×1024



lowpass filtering the pattern with a Gaussian kernel of size 21×21 , with $\sigma = 3.5$, $K=1$



Result of using a kernel of size 43×43 , with $\sigma = 7$, $K=1$



Recall: Gaussian Filters



Gaussian kernels of size 43×43 ,
with $\sigma = 7$



Kernel of 85×85 , with $\sigma = 7$



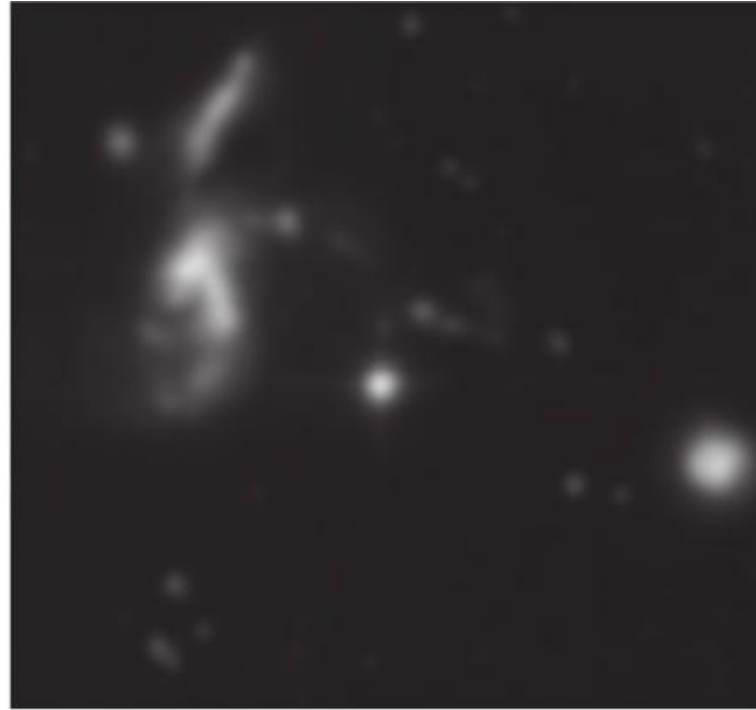
Difference image

Recall: Gaussian Filters - Example Usage

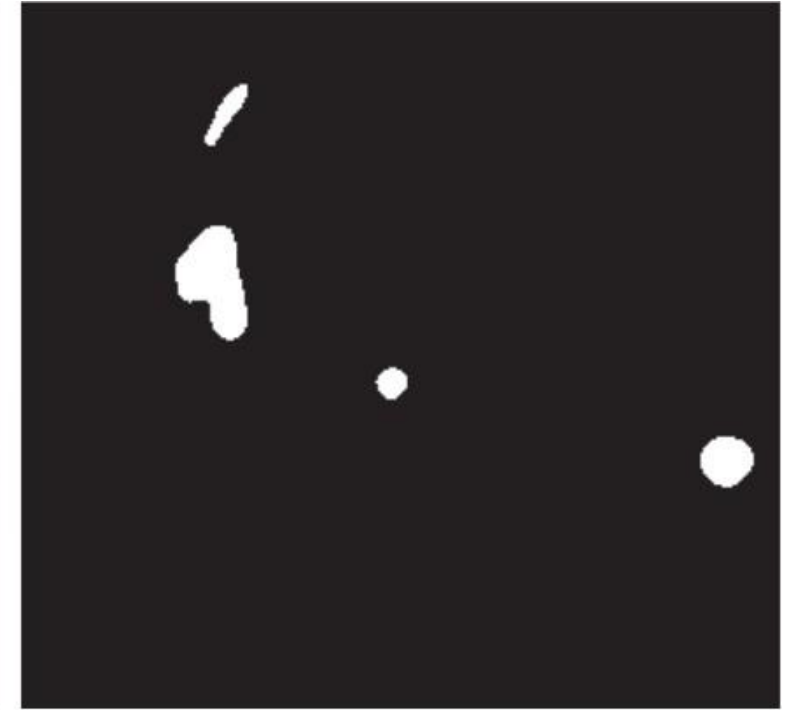


a b c

A 2566×2758 Hubble Telescope image of the Hickson Compact Group



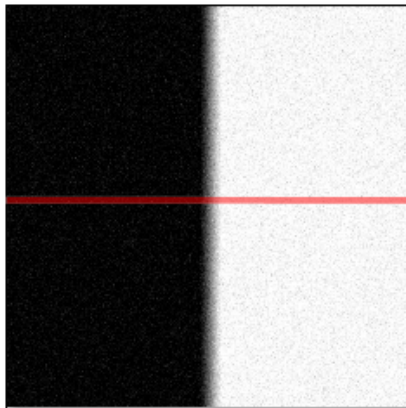
Result of lowpass filtering with a Gaussian kernel.



Result of thresholding the filtered image (intensities were scaled to the range $[0, 1]$)

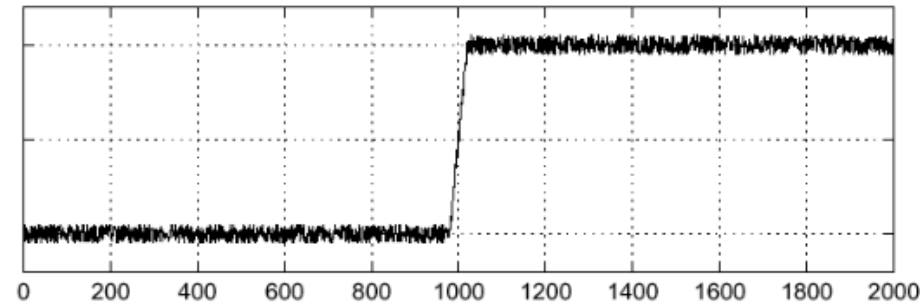
Effect of Noise

- Where is the edge?

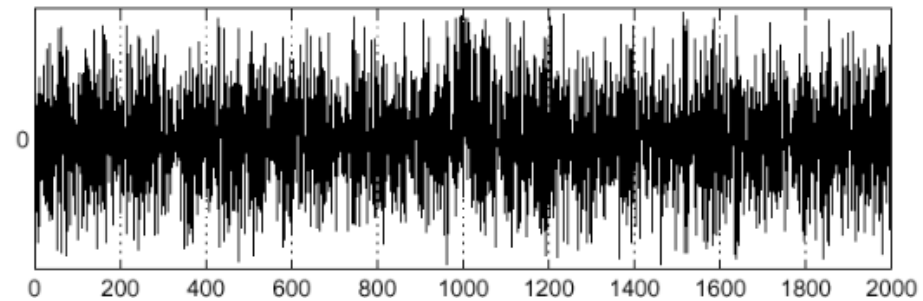


Noisy input image

$$f(x)$$

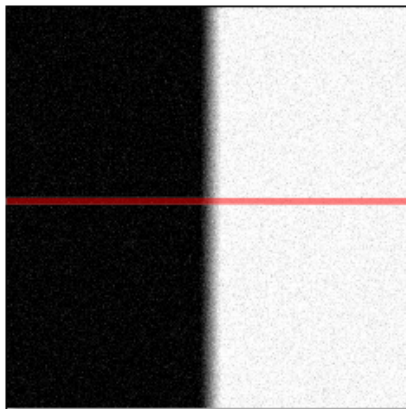


$$\frac{d}{dx}f(x)$$



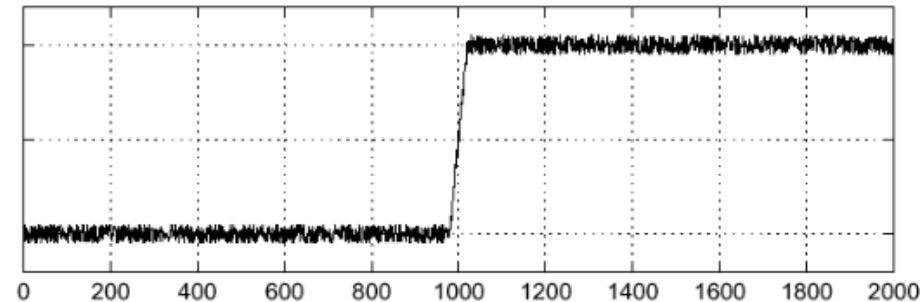
Effect of Noise

- Where is the edge?

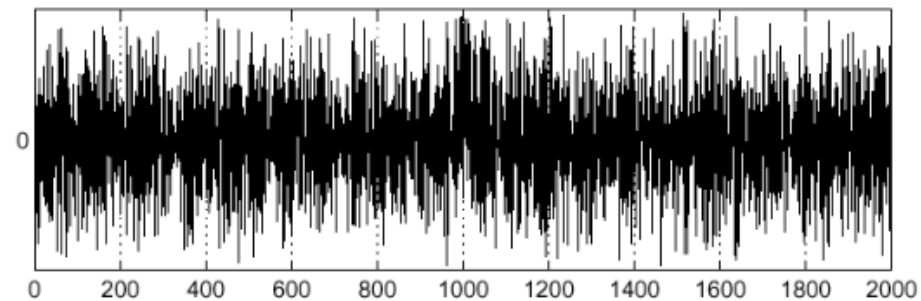


Noisy input image

$$f(x)$$



$$\frac{d}{dx}f(x)$$



Step -1: Smooth the image with gaussian low-pass filter



Step -2: Computing Gradients (Magnitude and Direction)

Compute partial derivatives $\partial f/\partial x$ and $\partial f/\partial y$ at every pixel location in the image

$$\nabla f(x, y) \equiv \text{grad}[f(x, y)] \equiv \begin{bmatrix} g_x(x, y) \\ g_y(x, y) \end{bmatrix} = \begin{bmatrix} \frac{\partial f(x, y)}{\partial x} \\ \frac{\partial f(x, y)}{\partial y} \end{bmatrix}$$

Points in the direction of maximum rate of change of f at (x, y)

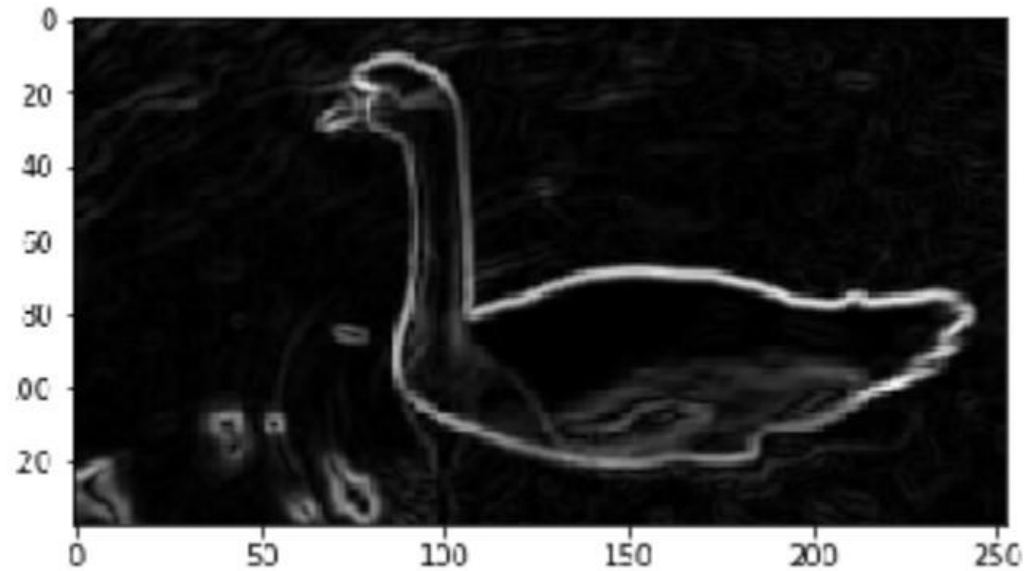
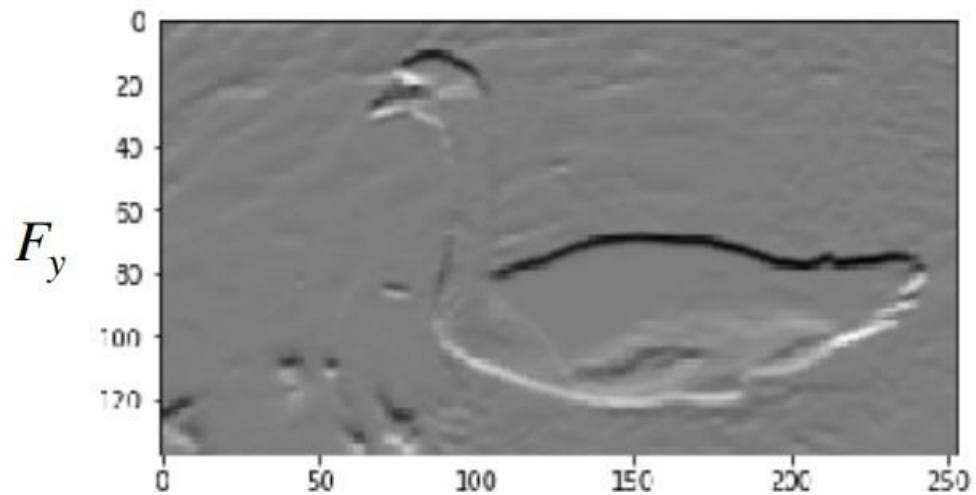
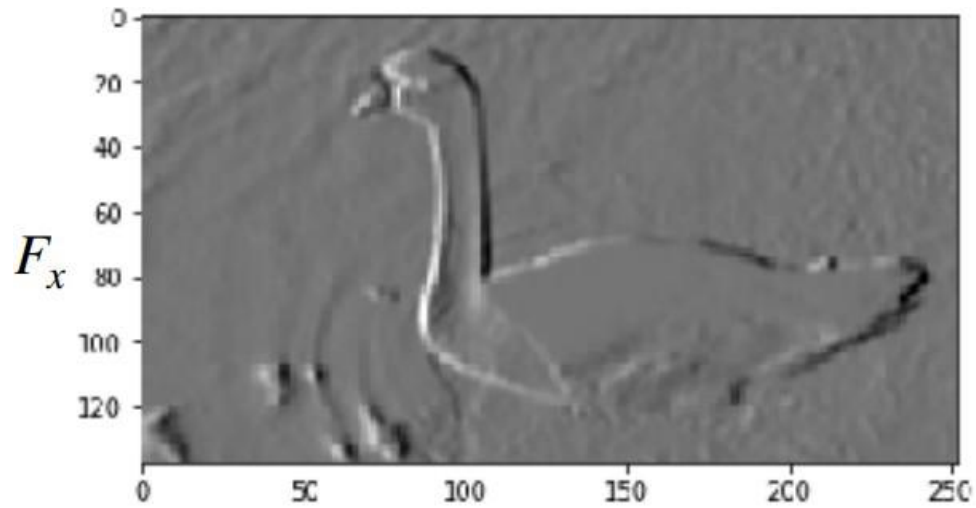
$$\alpha(x, y) = \tan^{-1} \left[\frac{g_y(x, y)}{g_x(x, y)} \right]$$

Direction of gradient vector at (x, y)

$$M(x, y) = \|\nabla f(x, y)\| = \sqrt{g_x^2(x, y) + g_y^2(x, y)}$$

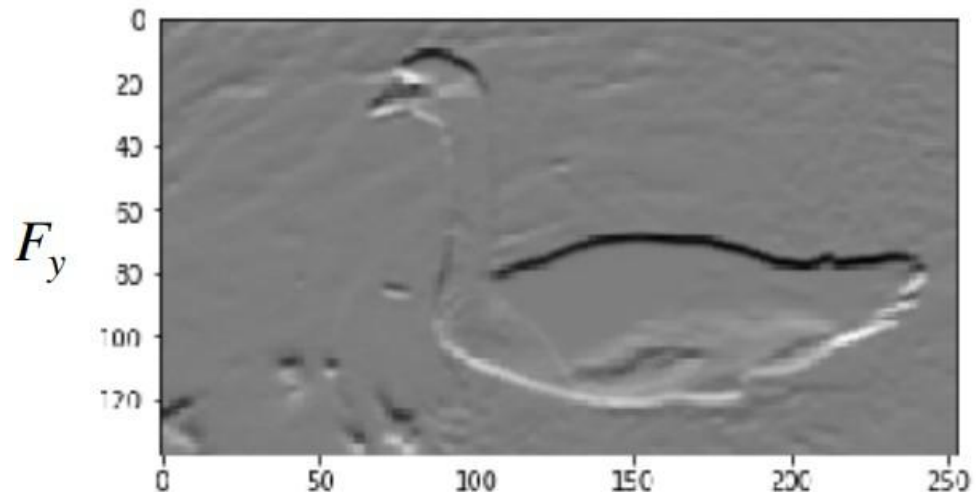
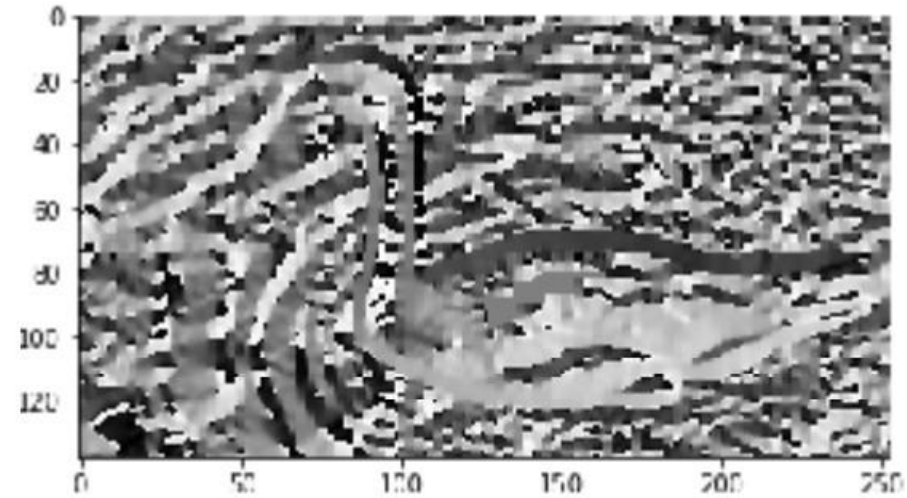
Value of the rate of change in the direction of the gradient vector at (x, y)

Step -2: Computing Gradients (Magnitude and Direction)



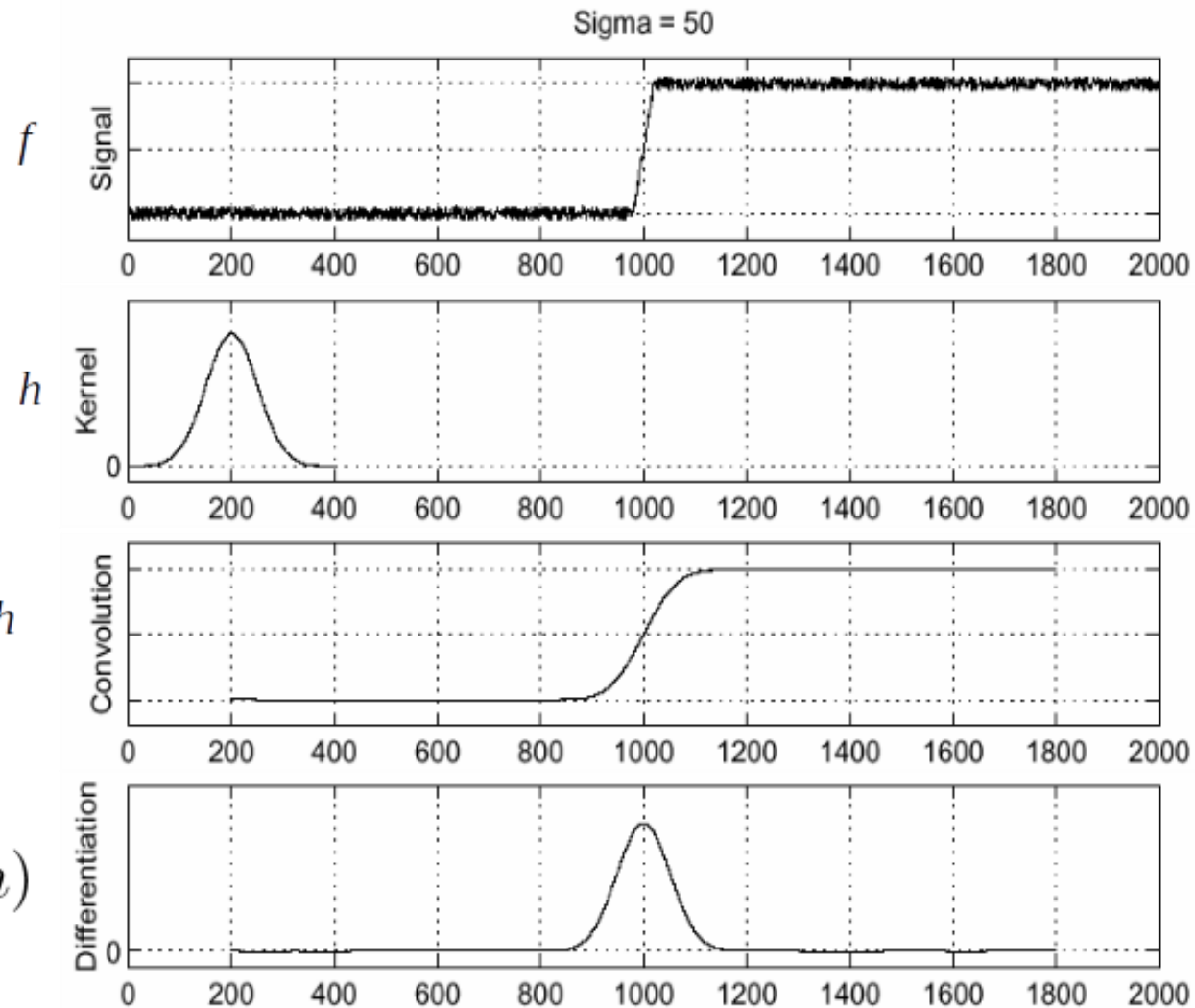
$$G = \sqrt{(F_x^2 + F_y^2)}$$

Step -2: Computing Gradients (Magnitude and Direction)



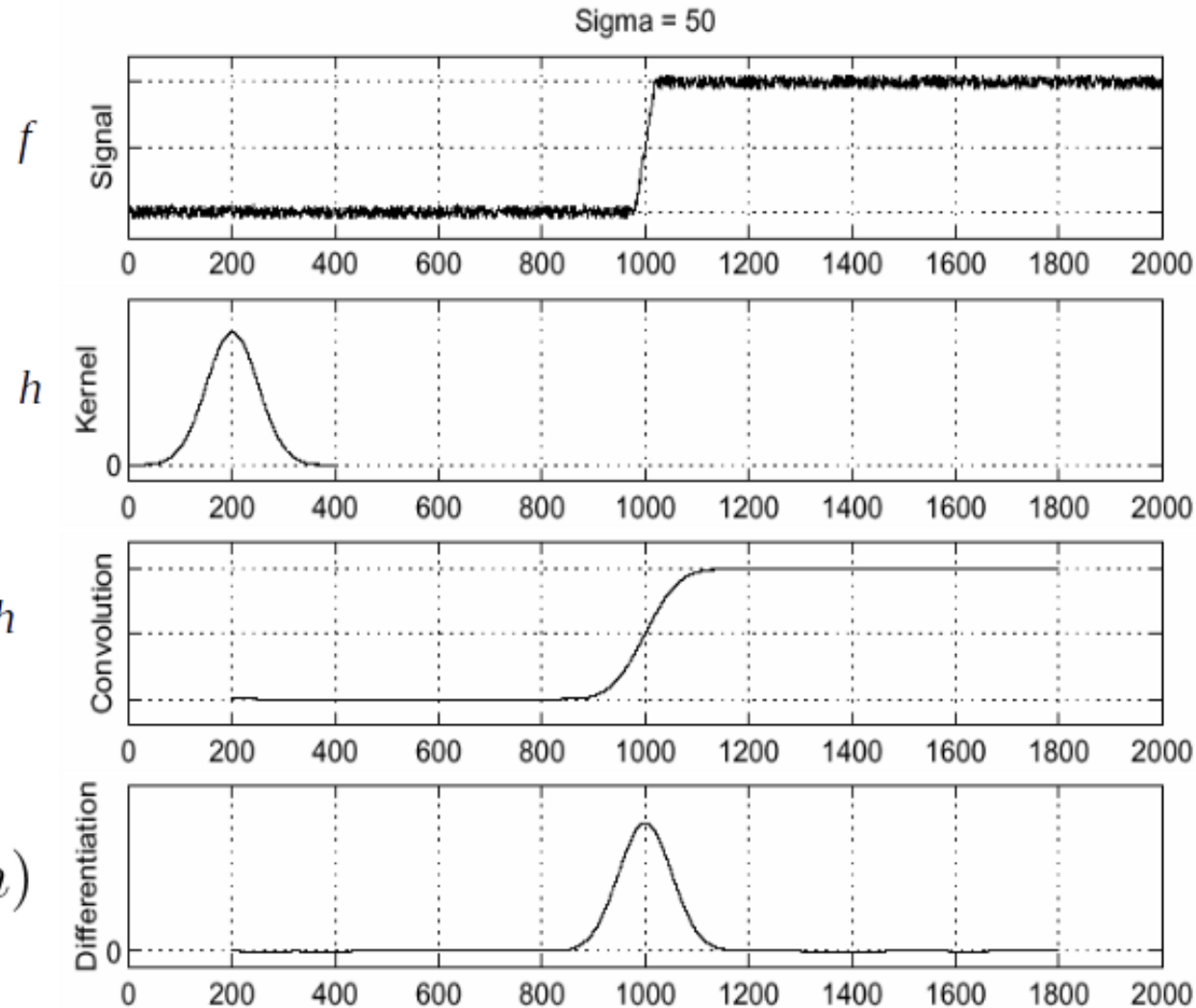
$$\theta = \tan^{-1} \left(\frac{F_y}{F_x} \right)$$

(Aside) Step -2: Efficient Computation of Image Gradients





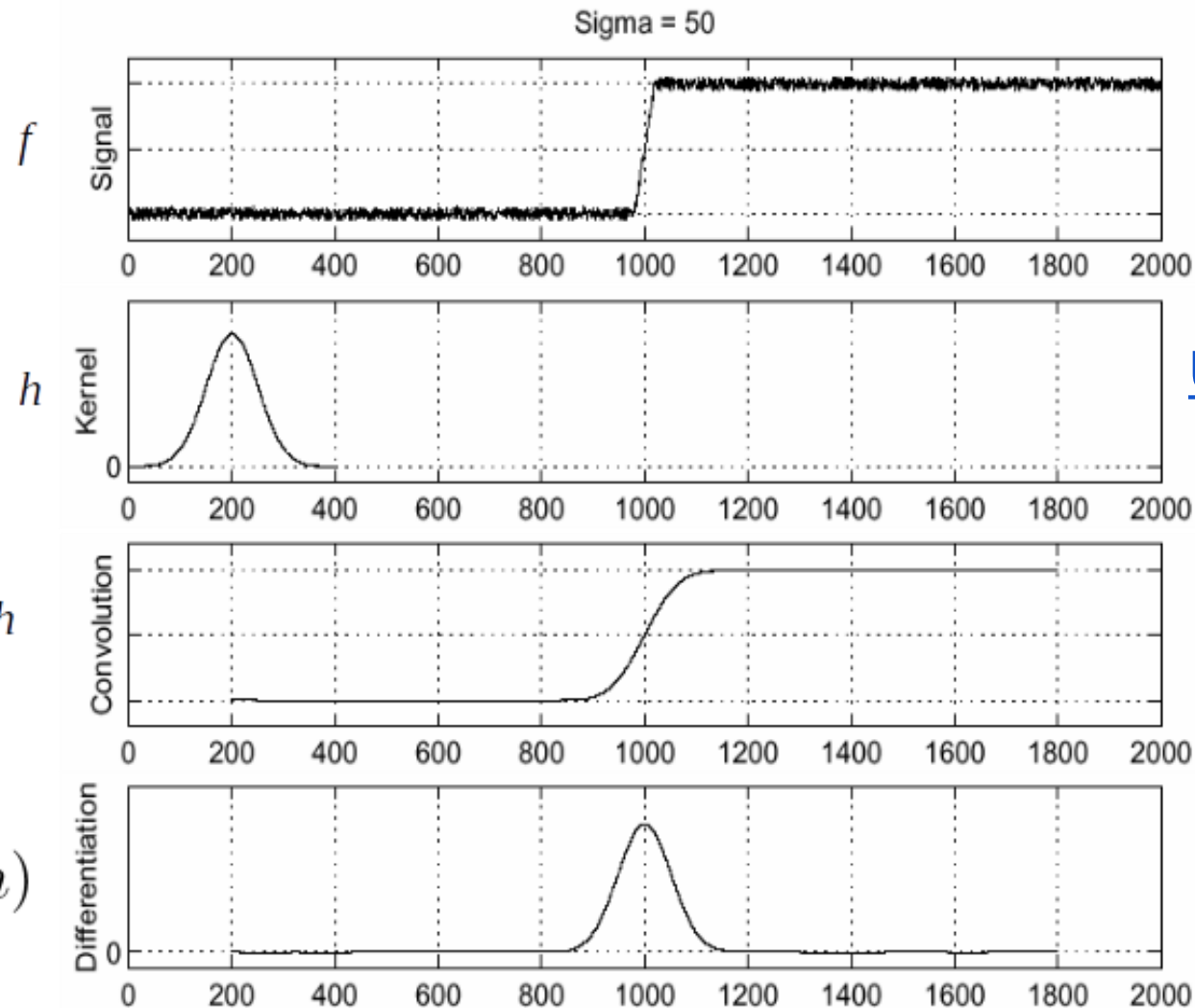
(Aside) Step -2: Efficient Computation of Image Gradients



Objective is to compute $\frac{d}{dx}(f * h)$



(Aside) Step -2: Efficient Computation of Image Gradients



Objective is to compute $\frac{d}{dx}(f * h)$

Using associative property of convolution

$$\frac{d}{dx}(f * h) = f * \frac{d}{dx}h$$

$$G_{\sigma}(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}}$$

$$G'_{\sigma}(x) = \frac{d}{dx}G_{\sigma}(x) = -\frac{1}{\sigma} \left(\frac{x}{\sigma}\right) G_{\sigma}(x)$$

This saves us one operation



Ex: Step -1 and Step-2



Original Image

Ex: Step -1 and Step-2

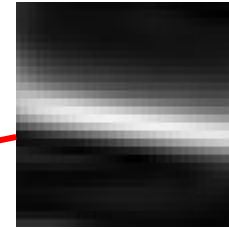


Smoothed Gradient Magnitude



Original Image

Ex: Step -1 and Step-2



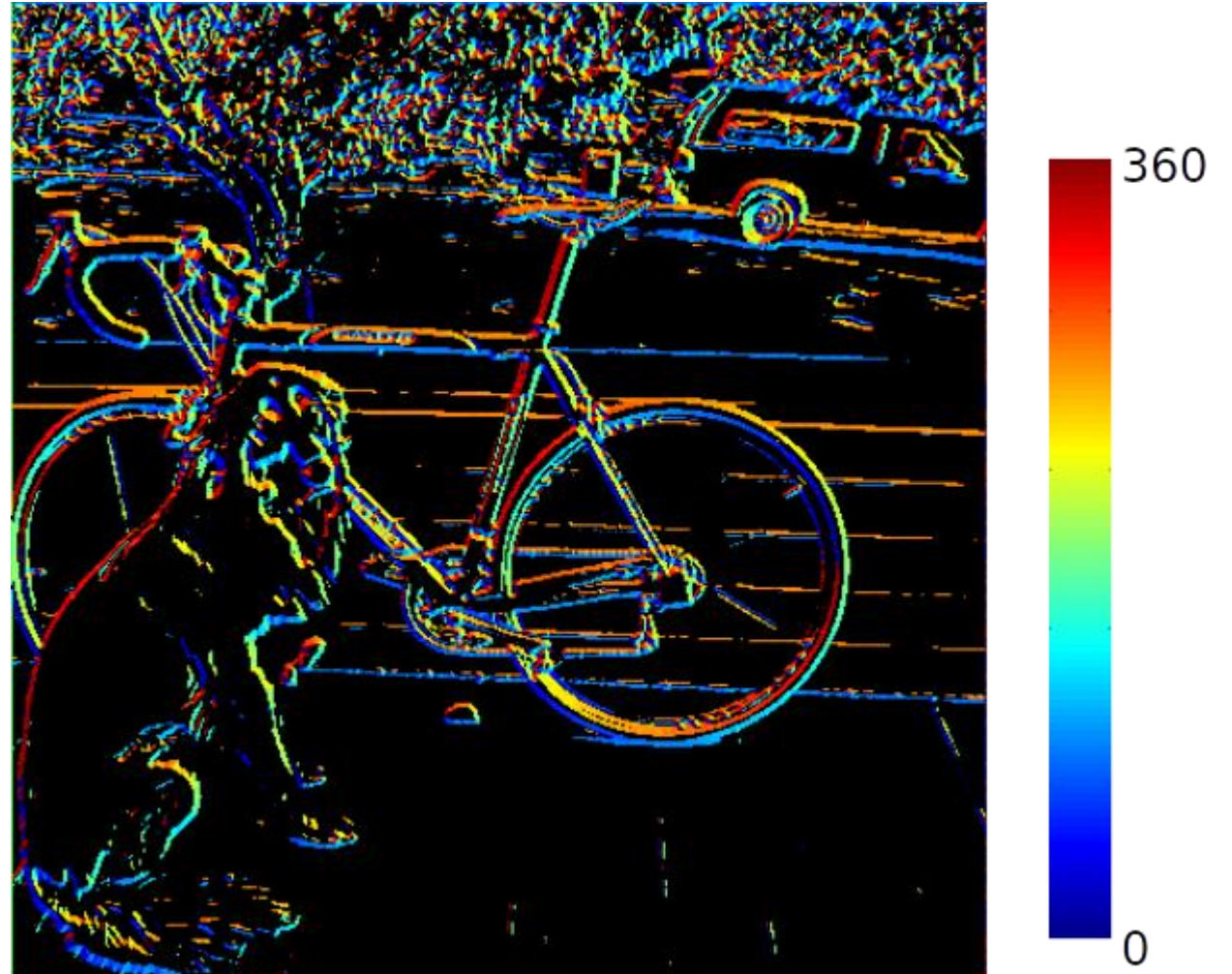
Where is the edge?

Smoothed Gradient Magnitude

Ex: Step -1 and Step-2



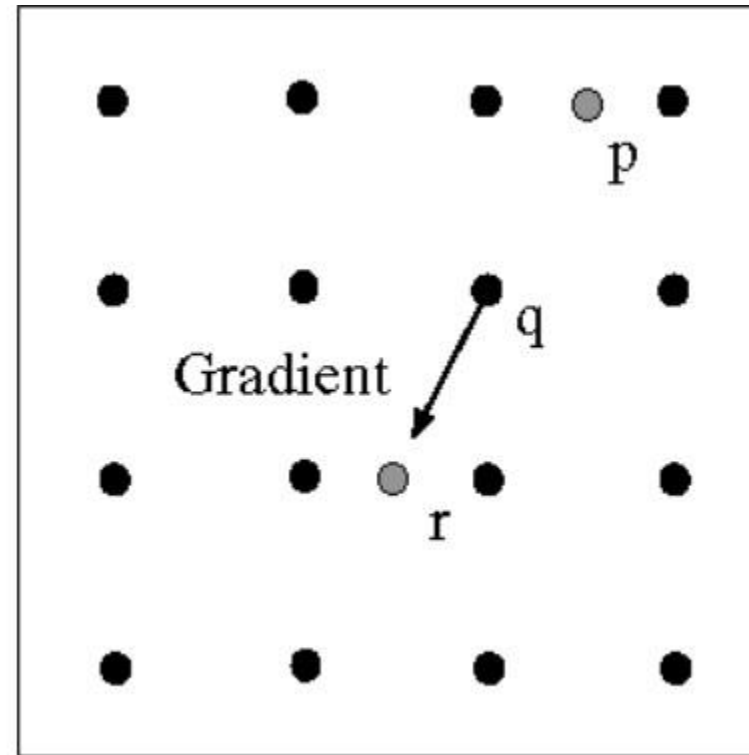
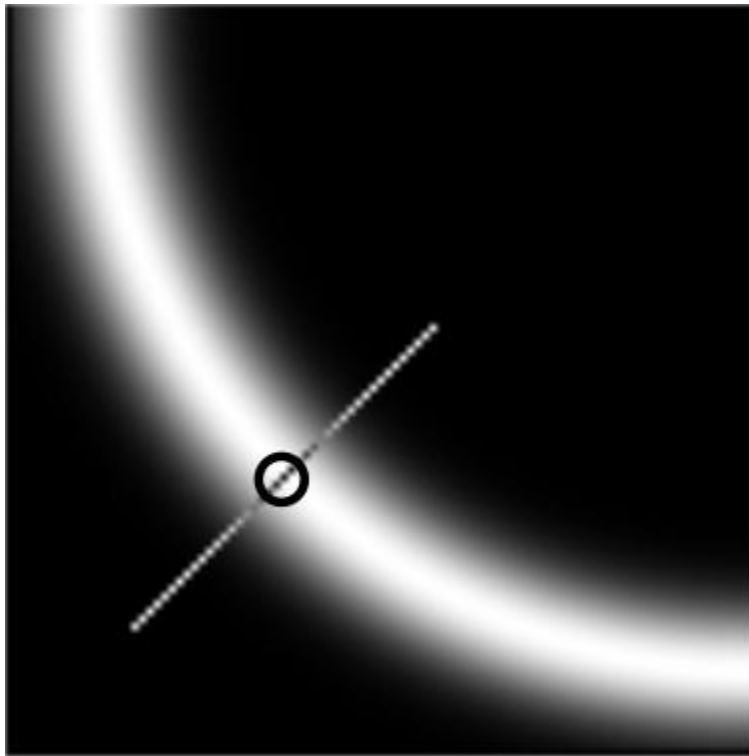
Smoothed Gradient Magnitude



Can we make use of angle to locate the edge better?

Step -3: Non-Maxima Suppression

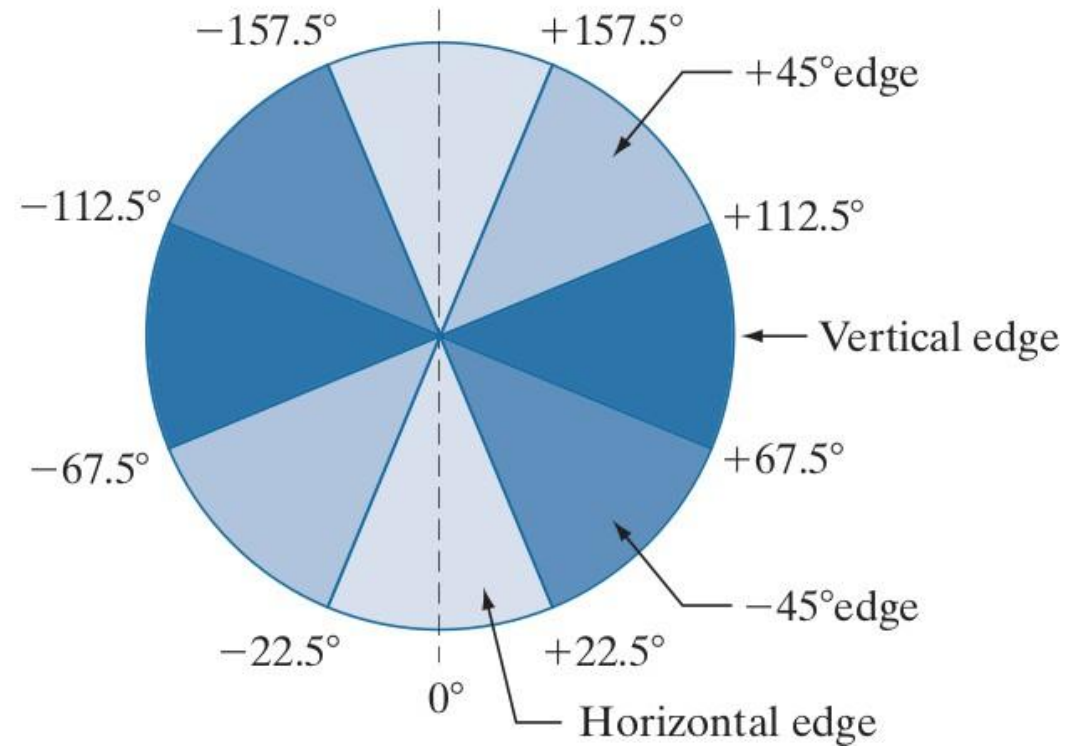
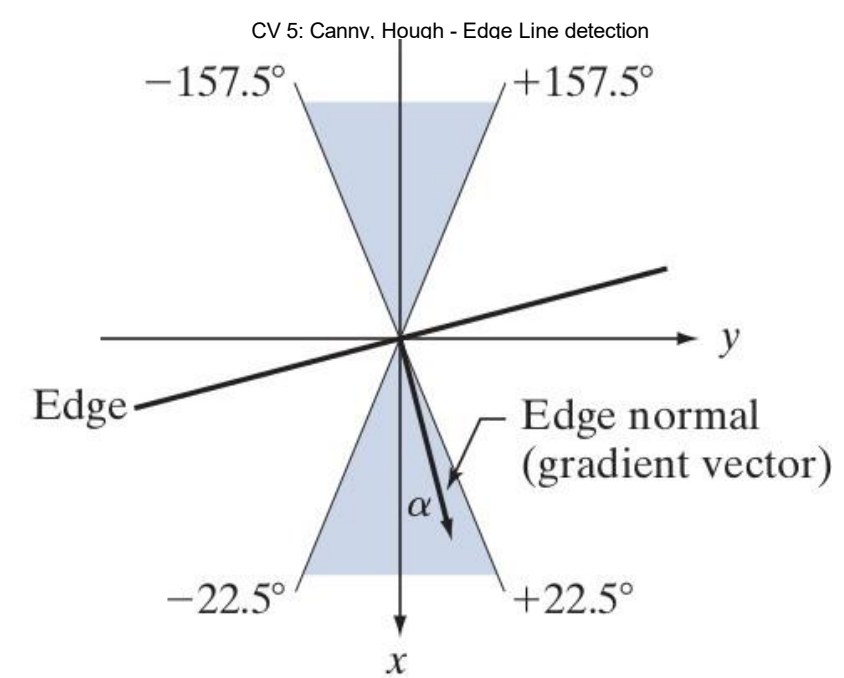
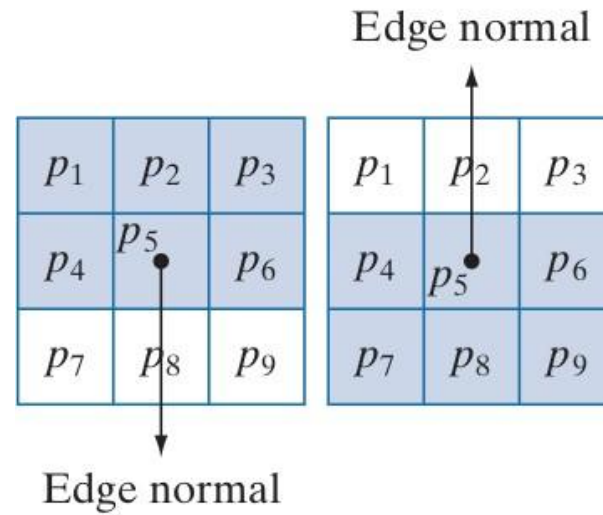
Check if pixel is local maximum along gradient direction





Step -3: Non-Maxima Suppression - Computation

Let d_1, d_2, d_3 and d_4 refers to horizontal, vertical, $+45^\circ$ and -45° edges.





Step -3: Non-Maxima Suppression - Computation

Let d_1, d_2, d_3 and d_4 refers to horizontal, vertical, $+45^\circ$ and -45° edges.

Let α be the matrix of gradient orientations

Consider a 3x3 neighborhood around $\alpha(x,y)$

Non-Maxima Scheme around $\alpha(x,y)$ is as below:

1. Find the direction d_k that is closest to $\alpha(x,y)$.
2. Let K denote the value of $\|\nabla f_s\|$ at (x,y) . If K is less than the value of $\|\nabla f_s\|$ at one or both of the neighbors of point (x,y) along d_k , let $g_N(x,y) = 0$ (suppression); otherwise, let $g_N(x,y) = K$.

Repeat (1) and (2) for all values of x and y to get non-maxima suppressed image which is of the same size as original image.

Step -3: Non-Maxima Suppression - Computation



Before Non-Maxima Suppression



After Non-Maxima Suppression

Step -4: Double Thresholding and Edge Linking

Still some noise; Only need strong edges;

Called as *Hysteresis thresholding*

Use two thresholds T_L and T_H which results in two images;

Choose the ratio of high to low threshold in the range of 2:1 to 3:1.



Step -4: Double Thresholding and Edge Linking

$\leq$ lower threshold	lower threshold $<$ intensity $<$ upper threshold	$\geq$ upper threshold
irrelevant = 0	weak = low threshold	strong = 255



Step -4: Double Thresholding and Edge Linking

$\leq$ lower threshold	lower threshold $<$ intensity $<$ upper threshold	$\geq$ upper threshold
irrelevant = 0	weak = low threshold	strong= 255

$$g_{NH}(x, y) = g_N(x, y) \geq T_H$$

$$g_{NL}(x, y) = g_N(x, y) \geq T_L$$

$$g_{NL}(x, y) = g_{NL}(x, y) - g_{NH}(x, y)$$





Step -4: Double Thresholding and Edge Linking

$\leq$ lower threshold	lower threshold $<$ intensity $<$ upper threshold	$\geq$ upper threshold
irrelevant = 0	weak = low threshold	strong= 255

$$g_{NH}(x, y) = g_N(x, y) \geq T_H \quad \text{Strong Edge Pixels \& Valid Edge Pixels}$$

$$g_{NL}(x, y) = g_N(x, y) \geq T_L$$

$$g_{NL}(x, y) = g_{NL}(x, y) - g_{NH}(x, y) \quad \text{Weak Edge Pixels \& we need to identify valid edge pixels from this set.}$$



Step -4: Double Thresholding and Edge Linking

$\leq$ lower threshold	lower threshold < intensity < upper threshold	$\geq$ upper threshold
irrelevant = 0	weak = low threshold	strong= 255

$$g_{NH}(x, y) = g_N(x, y) \geq T_H \quad \text{Strong Edge Pixels \& Valid Edge Pixels}$$

$$g_{NL}(x, y) = g_N(x, y) \geq T_L$$

$$g_{NL}(x, y) = g_{NL}(x, y) - g_{NH}(x, y) \quad \text{Weak Edge Pixels \& we need to identify valid edge pixels from this set.}$$



Step -4: Double Thresholding and Edge Linking

Approach to Link Edges:

- Strong edges are edges!
- Weak edges are edges iff they connect to strong
- Look in some neighborhood (usually 8 closest)



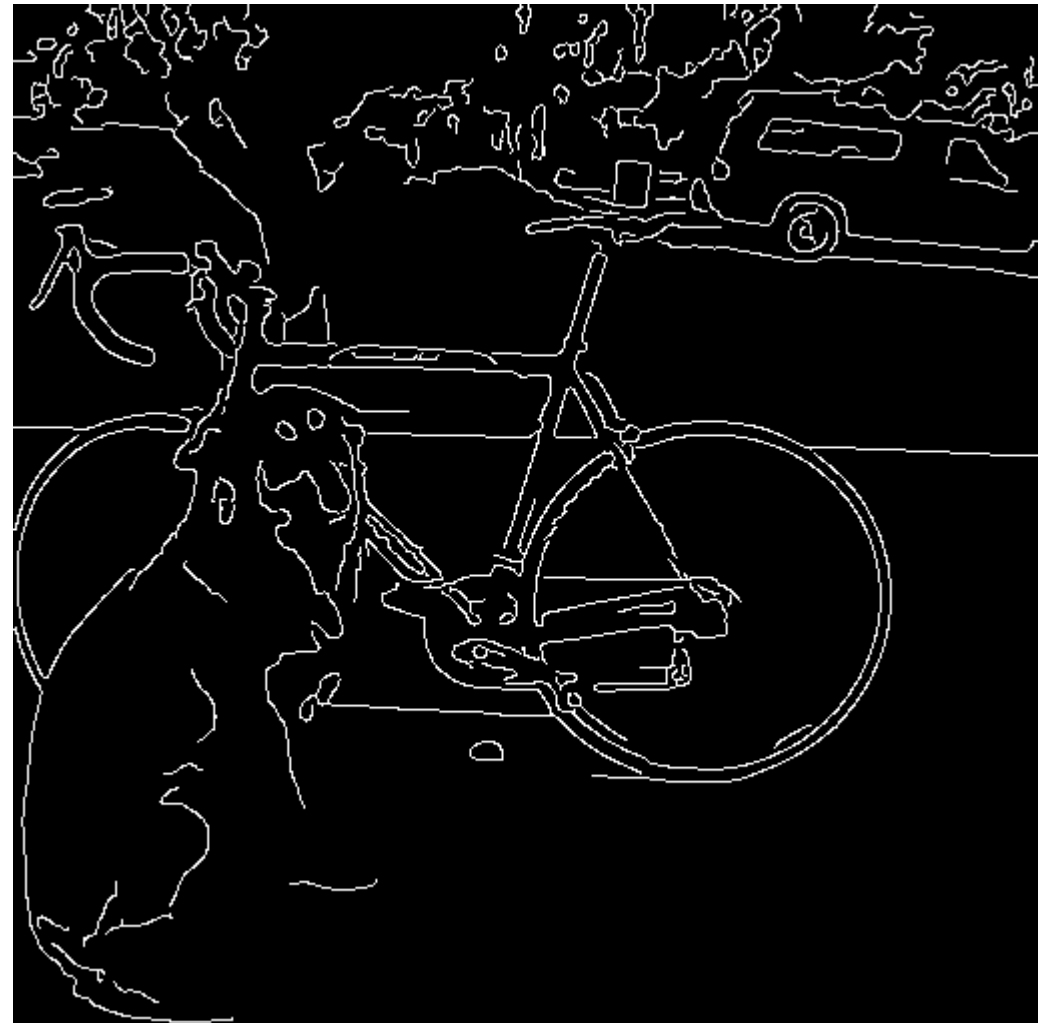
Step -4: Double Thresholding and Edge Linking

- (a)** Locate the next unvisited edge pixel, p , in $g_{NH}(x, y)$.
- (b)** Mark as valid edge pixels all the weak pixels in $g_{NL}(x, y)$ that are connected to p using, say, 8-connectivity.
- (c)** If all nonzero pixels in $g_{NH}(x, y)$ have been visited go to Step (d). Else, return to Step (a).
- (d)** Set to zero all pixels in $g_{NL}(x, y)$ that were not marked as valid edge pixels.

Step -4: Double Thresholding and Edge Linking



Before Edge Linking



After Edge Linking

Step -4: Double Thresholding and Edge Linking

Original
image



gap is gone



Strong +
connected
weak edges

Strong
edges
only



Weak
edges





Canny Edge detection algorithm

The Canny edge detector is the "Gold Standard" of edge detection. Developed by John F. Canny in 1986, it's a multi-stage process designed to find edges while suppressing noise. It's smarter than basic filters because it doesn't just look for color changes; it looks for the *best* edges.

The algorithm follows five distinct steps:

1. Noise Reduction (Gaussian Blur)

Edges are sensitive to image noise (tiny random pixels). If you don't smooth the image first, the algorithm will mistake noise for edges. We use a **Gaussian filter** to lightly blur the image.

2. Gradient Calculation

The algorithm finds where the intensity of the image changes most drastically. It calculates the **gradient intensity** and **direction** for every pixel, usually using a Sobel kernel.

- **Intensity:** How "strong" the edge is.
- **Direction:** Which way the edge is pointing (e.g., horizontal, vertical, or diagonal).

3. Non-Maximum Suppression

After step 2, the edges are often "thick" and blurry. This step thins them out. The algorithm looks at each pixel and checks if it is a local maximum in the direction of the gradient.

- If the pixel is the brightest in its immediate neighborhood along that line, it stays.
- If not, it's suppressed (set to zero).
- **Result:** A binary image with thin, one-pixel-wide lines.

4. Double Thresholding

Not all remaining edges are "real" edges. Some might be caused by remaining noise or slight color variations. Canny uses two thresholds:

- **High Threshold:** Anything above this is a "strong" edge (certainly an edge).
- **Low Threshold:** Anything below this is discarded.
- **In-between:** These are "weak" edges that are only kept if they meet the criteria in the next step.

5. Edge Tracking by Hysteresis

This is the final "cleanup." A weak edge pixel is only preserved if it is **connected** to a strong edge pixel. The logic is that true edges are continuous lines, while noise-induced edges are usually isolated.

Step	Action	Purpose
Gaussian Blur	Smoothing	Remove noise.
Gradient	Sobel Filter	Find intensity and direction.
Non-Max Suppression	Thinning	Make edges 1-pixel wide.
Hysteresis	Thresholding	Keep strong edges; link weak ones.

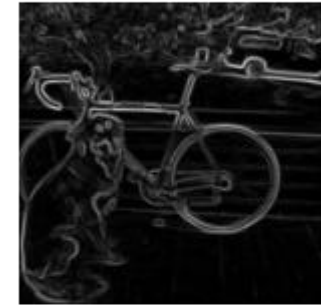


Summary of Canny Edge Detector

(1) Filter image with derivative of Gaussian



(2) Find magnitude and orientation of gradient



(3) Non-maximum suppression



(4) Linking and thresholding (hysteresis):

Define two thresholds: low and high

Use the high threshold to start edge curves and
the low threshold to continue them

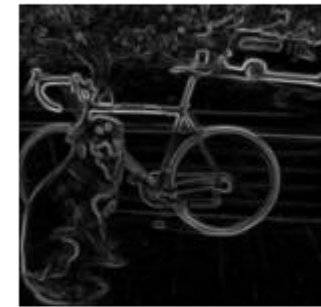




Summary of Canny Edge Detector

J. Canny, **A Computational Approach To Edge Detection**,
IEEE Trans. Pattern Analysis and Machine Intelligence, 8:679-
714, 1986.

Our first computer vision pipeline!
Still a widely used edge detector in computer vision





- Readings:

(1) Hough Transform - Page 737 - 742 - Digital Image Processing, 4th Ed, Rafael C. Gonzalez & Richard E. Woods

- Next Class:

Image Sharpening; Linear Filters; Edge Detection using Canny; Line detection using Hough transforms; [if possible]